

PROJECT REPORT

on

INTELLIGENT EMAIL THREAT DETECTION AND USER AWARENESS SYSTEM

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE (Artificial Intelligence & Machine Learning)

Submitted by

E. Akhila (23UP1A6669)

G. Sindhu (23UP1A6671)

Karnika Reddy (23UP1A6678)

N. Greeshma (23UP1A6688)

Under the Guidance of

Mrs. G. Ramya (M.Tech)

Assistant Professor



DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

VIGNAN 'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY FOR WOMEN

Accredited to NBA NAAC A+(CSE&ECE)

(An Autonomous Institution) Approved by AICTE and Affiliated to JNTUH Kondapur

(Village), Ghatkesar (Mandal), Medchal (Dist.), Telangana, Pincode-501301

www.vmtw.edu.in

2025-2026



VIGNAN'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY FOR WOMEN

(An Autonomous Institution)

[Sponsored by Lavu Educational Society, Affiliated to JNTUH & Approved by AICTE, New Delhi]

Kondapur (V), Ghatkesar (M), Medchal - Malkajgiri (D) - 501 301. Phone: 96529 10002/3



DEPARTMENT OF CSE(AI&ML)

CERTIFICATE

This is to certify that project work entitled “INTELLIGENT EMAIL THREAT DETECTION AND USER AWARENESS SYSTEM” submitted by E. Akhila(23UP1A6669), G.Sindhu (23UP1A6671), Karnika Reddy(23UP1A6678), N. Greeshma(23UP1A6688) in the partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in CSE(AI&ML) VIGNAN'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY FOR WOMEN is a record of Bonafide work carried by them under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or institute for the award of any degree.

PROJECT GUIDE

Mrs. G. RAMYA

(M.Tech)

Assistant Professor

PROJECT COORDINATOR

Mrs. G. RAMYA, P.PAVANI

(M.Tech)

Assistant Professor

HEAD OF DEPARTMENT

Dr. M. VISHNU VARDHANA RAO

(Ph. D)

Associate Professor

EXTERNAL EXAMINAR



VIGNAN'S INSTITUTE OF MANAGEMENT AND TECHNOLOGY FOR WOMEN

(An Autonomous Institution)

[Sponsored by Lavu Educational Society, Affiliated to JNTUH & Approved by AICTE, New Delhi]

Kondapur (V), Ghatkesar (M), Medchal - Malkajgiri (D) - 501 301. Phone: 96529 10002/3



DEPARTMENT OF CSE(AI&ML)

DECLARATION

We here by declare that the work reported in the present project entitled “**INTELLIGENT EMAIL THREAT DETECTION AND USER AWARENESS SYSTEM**” is a record of Bonafide work duly completed by us in the Department of CSE (AI&ML) from Vignan’s Institute of Management and Technology for Women, affiliated to JNTU, Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. All such materials that have been obtained from other sources have been duly acknowledged. The result embodied in this project report have not been submitted to any other University or Institute for the award of any degree to the best of our knowledge and belief.

S.no	Roll Number	Student Name	Signature of the Student
01	(23UP1A6669)	E. Akhila	
02	(23UP1A6671)	G. Sindhu	
03	(23UP1A6678)	Karnika Reddy	
04	(23UP1A6688)	N. Greeshma	

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and indebtedness to our project guide **Mrs. G. Ramya (M.Tech)**, Assistant Professor, Department of CSE (Artificial Intelligence and Machine Learning) from Vignan's Institute of Management and Technology for women, for his valuable suggestions and interest throughout the course of this project.

We would like to extend our gratitude to **Mrs. G. Ramya (M.Tech)**, **Mrs.P.Pavani(M.Tech)** Project Coordinator, Department of CSE(Artificial Intelligence and Machine Learning) from Vignan's Institute of Management and Technology For Women, for her valuable suggestions and timely help while the project. We are also thankful to **Dr. M. Vishnu Vardhana Rao (Ph. D)** Head of the Department of CSE - Artificial Intelligence and Machine Learning from Vignan's Institute Of Management And Technology For Women, Hyderabad for providing an excellent infrastructure and a nice atmosphere for completing this project successfully as a part of our B.Tech CSE(AI&ML).

We convey our heartfelt thanks to the lab staff for allowing us to use the required equipment whenever needed.

Finally, we would like to take this opportunity to thank our family for their support through the work. We sincerely acknowledge and thank all those who gave directly or indirectly their support in the completion of this work.

- | | |
|------------------------|---------------------|
| 1.E. Akhila | (23UP1A6669) |
| 2.G. Sindhu | (23UP1A6671) |
| 3.Karnika Reddy | (23UP1A6678) |
| 4.N. Greeshma | (23UP1A6688) |

ABSTRACT

In today's digital era, email has become an essential medium of communication. However, it also increases the risk of cyber threats such as spam, phishing, and various malicious attacks. Many users find it difficult to identify whether an email is safe or harmful, which often leads to serious issues like data theft, financial loss, and privacy problems. Traditional email filtering systems simply label messages as "spam" without explaining the reason, and they are not strong enough to detect new and sophisticated threats. Therefore, there is a need for an advanced system that can not only detect risks but also create awareness among users.

The proposed system, is an intelligent email threat detection and awareness platform. It classifies emails into three categories: Safe, Spam, and Phishing. The system uses machine learning techniques to analyse different parts of an email, including the subject, content, and URL links. During preprocessing, unnecessary elements are removed, and the data is converted into meaningful features using techniques like TF-IDF. Then, algorithms such as Logistic Regression and Naïve Bayes are applied to classify emails accurately based on patterns and suspicious characteristics. This helps in identifying hidden threats and improving detection performance.

A key feature of this system is the "Insight and Awareness" module. Instead of just displaying the result, it explains why an email is considered risky and provides suggestions for safe actions. Through a user-friendly web interface, users can get instant results, warnings, and safety tips. Overall, this project combines threat detection with user awareness, making email communication more secure and helping users better understand and respond to cyber threats.

TABLE OF CONTENTS

CONTENTS	PAGE NO
1.INTRODUCTION	1
1.1 About the Problem	1
1.2 About the project documentation and its specifications	2
1.3 Literature Survey	3
2.ANALYSIS	4
2.1 Description of the Problem	4
2.2 Objectives	5
2.3 Description of the existing System	7
2.4 Evaluation of the existing system	8
2.5 Description of Other possible solutions	9
2.6 Evaluation of the possible solution	10
3.DESIGN	13
3.1 Project Plan	13
3.2 Description of Method of Solution	13
3.3 System Architecture	14
3.4 UML Diagrams	16
3.5 Hardware Requirements	22
3.6 Software Requirements	23
4.IMPLEMENTATION	24
5.TESTING AND DOCUMENTATION	46
5.1 Test strategy	46

5.2 Test Results	46
5.3 Technical Documentation	47
5.4 User Documenting	47
6. CONCLUSION	48
6.1 Evaluation	48
6.2 Future Enhancement	49
6.3 Technical Skills Learned	50
7. REFERENCES	51
8. BIBLIOGRAPHY	52
9. APPENDIX	53
9.1 Screenshots	53
9.2 Dataset	57
9.3 Monthly Reports	58

LIST OF FIGURES

S. NO	FIGURE NAME	PAGE NO
Fig 3.3	System Architecture	14
Fig 3.4.1	Use case diagram	16
Fig 3.4.2	Class diagram	17
Fig 3.4.3	Sequence Diagram	18
Fig 3.4.4	Activity Diagram	19
Fig 3.4.5	Deployment Diagram	20

1. INTRODUCTION

1.1 About the problem

In today's fast-paced digital world, email has become an essential communication tool for both personal and professional use. However, along with its convenience comes a growing number of cyber threats such as spam, phishing attacks, and malicious attachments. These threats are becoming increasingly sophisticated and are often designed to look genuine, making it difficult for users to identify whether an email is safe or harmful. As a result, many individuals unintentionally click on unsafe links or download infected files, which can lead to serious issues like data theft, financial loss, and unauthorized access to confidential information.

Traditional email security systems are no longer sufficient to handle these advanced threats. Most existing solutions rely on simple rule-based filtering or keyword detection, which can easily be bypassed by modern attackers. They often fail to detect newly emerging or cleverly disguised malicious emails. Another major concern is the lack of user awareness. Current systems usually label emails as "spam" or "not spam" without explaining the reason behind the classification. This lack of transparency leaves users confused and unaware of the actual risks, causing them to ignore warnings or make unsafe decisions.

Another significant challenge in email security is the rapid evolution of cyber attack techniques. Cybercriminals continuously update their strategies to bypass existing security measures, using advanced methods such as social engineering, spoofed email addresses, and personalized phishing messages. These attacks often target human emotions like urgency, fear, or curiosity, making them even harder to detect through traditional filtering systems. As attackers become more intelligent, static and rule-based systems struggle to keep up, creating a gap between current security capabilities and real-world threats.

In addition, organizations and individuals face difficulties in managing the increasing volume of emails received. With a large number of emails to process daily, users tend to make quick decisions without carefully analysing each message. This increases the likelihood of overlooking warning signs in suspicious emails. The absence of an intelligent system that can assist users in real-time, while also providing meaningful insights, further worsens the problem. Therefore, addressing scalability and real-time threat detection, along with user support, has become an essential requirement in modern email security systems.

Therefore, there is a clear need for a more advanced and intelligent system that not only detects harmful emails with higher accuracy but also educates users about potential threats. A solution that combines strong detection mechanisms with user awareness can significantly reduce cyber risks and promote safer email usage.

1.2 About the project Documentation and its specifications

This project proposes the development of an intelligent email threat detection system that focuses on both accurate classification and user guidance. The system is designed using machine learning techniques, which allow it to learn from past data and recognize patterns associated with different types of emails such as safe, spam, and phishing. By analysing various components of an email—including the subject line, message content, sender details, and embedded links—the system can make informed predictions about its safety.

The implementation involves several key stages, starting with data collection from relevant email datasets. This is followed by preprocessing steps such as removing unnecessary words, cleaning the text, and extracting meaningful features. These features are then used to train classification algorithms that can accurately identify the nature of incoming emails. The system is designed to continuously improve its performance as more data becomes available, making it adaptable to evolving cyber threats.

An important aspect of this project is its focus on user interaction and awareness. Unlike traditional systems, this solution provides clear explanations for its predictions, helping users understand why an email is marked as suspicious or safe. It also offers practical suggestions, such as avoiding unknown links, checking sender authenticity, and being cautious with attachments. This makes the system not only a detection tool but also an educational platform.

From a technical perspective, the project includes well-defined modules such as data preprocessing, feature extraction, model training, prediction, and user interface design. The system can be developed using programming languages like Python along with machine learning libraries and basic web technologies for the interface. It requires a standard computing environment capable of handling data processing and model execution efficiently.

Overall, this project aims to create a reliable, efficient, and user-friendly email security solution. By integrating intelligent detection with user education, it enhances both system performance and user confidence, ultimately contributing to a safer digital communication environment.

1.3 Literature Survey

Name of the paper,author	Year	Category	Description	Key Insights	Limitations	References
Phishing Email Detection Using ML. R.Verma & N.Hossain	2019	ML,Cyber Security	Uses ML algorithm Like Naïve Bayes & SVM to classify phishing,legitimate Emails based on content.	ML-based detection improves accuracy compared to traditional rule-based methods.	Does not include user awareness feedback loop,realtime User alerts.	<u>Phishing Email Detection-THEMIS</u>
Spam, Malicious Email Detection Using NLP. S.Gupta & A.Kumar	2020	NLP	Applies NLP techniques to analyze email text, keywords, Sentence Patterns to detect malicious intent.	Context based analysis improves identification of hidden threats.	Performance may decrease with obfuscated or new phishing variants.	<u>Hybrid Phishing Emails Detection (MDPI review)</u>
URL Based phishing detection systems. J.Ma, L.Huang	2021	Cyber Security	Detects phishing emails by analysing Embedded URLs using features like domain age, URL length.	Effectively detects phishing before users click malicious links.	Less effective for emails without URLs.	<u>Detection of URL-based phishing Attacks using ML-IJERT 2019</u>
Intelligent email threat detection using hybrid models. K.Singh, P.Sharma	2023	ML&AI	Combines ML,NLP,rule based approaches with threat severity alerts for users.	Hybrid models provide higher accuracy and adaptability.	Higher system complexity and computational cost.	<u>HELPED: Hybrid Ensemble Learning Phishing Email Detection</u>

2. ANALYSIS

2.1 Description of the problem

We're In recent years, email has become one of the most essential communication platforms for individuals, organizations, and businesses. However, its widespread usage has also made it a primary target for cybercriminals. Threats such as spam emails, phishing attacks, malware distribution, and social engineering techniques are increasingly common. These malicious emails are often carefully designed to appear legitimate by imitating trusted organizations, using convincing language, and including realistic-looking links or attachments. As a result, users frequently fail to identify these threats and unknowingly expose sensitive information.

One of the major problems is the lack of user awareness and technical knowledge. Many users are unable to analyze email content critically, especially when the message creates urgency or fear. For example, emails claiming account suspension or urgent payment requests often lead users to act quickly without verifying authenticity. This behavior increases the chances of falling victim to cyber attacks. Additionally, phishing emails are becoming more sophisticated by using personalized information and advanced obfuscation techniques, making detection even more difficult.

Another critical issue is the limitation of traditional email filtering systems. These systems mostly depend on static rules, keyword matching, or blacklisting techniques. While they may successfully filter basic spam, they are not effective against modern, evolving threats. They often fail to analyze the actual intent or context of the email. Furthermore, these systems do not provide explanations or guidance to users, leaving them unaware of potential risks. This gap highlights the need for an intelligent, adaptive, and user-focused solution that can both detect threats and educate users.

Another growing concern is the increasing use of mobile devices for accessing emails. Many users now check their emails on smartphones or tablets, where screen size limitations make it harder to closely inspect sender details, URLs, or attachments. Important warning signs, such as slight changes in email addresses or suspicious links, often go unnoticed on smaller screens. This convenience of mobile access, while beneficial, unintentionally creates more opportunities for attackers to exploit user inattention and increase the success rate of phishing attempts.

Moreover, organizations face serious challenges in maintaining consistent email security across all users. Even if advanced security tools are implemented at the organizational level, human error remains one of the weakest links. A single careless action, such as clicking on a malicious link, can compromise an entire system or network. This highlights that technology alone is not enough—there is a strong need to combine technical solutions with user-focused awareness and training to build a more secure environment.

In addition to security risks, there is also a lack of transparency in how existing systems classify emails. Most email platforms simply move suspicious messages to spam folders without providing detailed explanations. Users are left wondering why a particular email was flagged or missed, which reduces their trust in the system. Without clear feedback or reasoning, users cannot learn from their mistakes or improve their ability to identify threats independently. This lack of interpretability limits the overall effectiveness of current solutions.

Finally, the increasing volume of daily email communication makes manual verification nearly impossible. Users often receive dozens or even hundreds of emails each day, making it unrealistic to carefully evaluate each message. This leads to decision fatigue, where users may ignore potential warning signs due to time pressure or overload. As a result, there is a strong demand for an intelligent system that can automate threat detection efficiently while also assisting users in making informed decisions. Such a system should not only identify risks but also simplify complex information into understandable insights for everyday users.

2.2 Objectives

The main objective of this project is to develop an advanced email threat detection system that not only strengthens communication security but also enhances user awareness. In today's digital environment, where cyber threats are becoming more frequent and sophisticated, it is essential to build a system that goes beyond basic filtering. This project focuses on creating a solution that combines intelligent detection techniques with a user-friendly approach, ensuring that even non-technical users can understand and benefit from it. The goal is to make email communication safer while also educating users about potential risks.

One of the primary objectives of this system is to automatically classify emails into categories such as Safe, Spam, or Phishing. This classification helps users quickly identify the nature of an email without needing to manually analyze it. By using automated processes, the system reduces the time and effort required to filter emails, making communication more efficient. At the same time, it ensures that potentially harmful emails are identified before they can cause any damage.

Another important objective is to apply machine learning techniques for analyzing email content and identifying hidden patterns. Unlike traditional rule-based systems, machine learning allows the system to learn from data and improve over time. It can detect subtle differences between legitimate and malicious emails by analyzing patterns in language, structure, and behavior. This makes the system more adaptable to new and evolving threats, providing a smarter and more reliable solution.

The project also aims to process different components of an email, such as the subject line, body content, and embedded URLs, to achieve accurate threat detection. Each part of an email carries valuable information that can indicate whether it is safe or suspicious. For example, certain keywords,

unusual links, or misleading subject lines can be strong indicators of phishing attempts. By analyzing these elements together, the system can make more informed and precise decisions.

Improving detection accuracy while reducing false positives and false negatives is another key objective of this project. False positives occur when safe emails are incorrectly marked as threats, while false negatives happen when harmful emails are not detected. Both situations can create problems for users, either by causing unnecessary confusion or by allowing dangerous emails to pass through. This project focuses on optimizing the system to strike a balance, ensuring reliable and consistent performance.

In addition to detection, the system is designed to provide meaningful explanations for its classification results. Instead of simply labeling an email as spam or phishing, it explains the reasons behind the decision. This helps users understand what makes an email suspicious, such as the presence of unknown links or unusual sender details. By offering transparency, the system builds trust and enables users to learn from each interaction.

The project also includes the objective of generating real-time alerts and safety recommendations for users. When a potentially harmful email is detected, the system immediately notifies the user and provides guidance on what actions to take. For example, it may suggest avoiding clicking on certain links or verifying the sender's identity. These real-time responses help users make safer decisions and reduce the chances of falling victim to cyber attacks.

Another important goal is to create an interactive and easy-to-use interface that is accessible to all users, including those with limited technical knowledge. The system is designed to present information in a simple and clear manner, making it easy for users to input email details and understand the results. A user-friendly interface ensures that the system is practical and can be widely adopted without requiring specialized skills.

The project also aims to minimize risks related to cyber fraud, data leakage, and unauthorized access. By accurately detecting threats and guiding users, the system acts as a protective layer against various types of cyber attacks. This not only safeguards individual users but also helps organizations maintain the security of their data and systems.

Finally, the system is designed to be scalable and adaptable for future use. As cyber threats continue to evolve, the system should be capable of updating itself with new patterns and techniques. This ensures long-term effectiveness and relevance. By building a flexible framework, the project aims to provide a solution that can grow and improve over time, meeting the changing demands of email security.

2.3 Description of the Existing Solution

Existing email security solutions are widely used by email service providers and organizations to protect users from unwanted or harmful messages. These systems form the first line of defence against threats such as spam, phishing, and malware. Traditionally, they rely on rule-based filtering techniques, where predefined rules are used to identify suspicious emails. For example, emails containing certain keywords, unusual formatting, or unknown sender addresses may be flagged as spam. In addition to this, blacklist and whitelist approaches are also commonly used. Blacklists block emails from known malicious sources, while whitelists allow emails only from trusted senders. These methods are simple and effective for basic filtering, but they have limitations when dealing with modern and more sophisticated threats.

To improve detection capabilities, many modern systems have started incorporating machine learning and Natural Language Processing (NLP) techniques. These advanced methods enable systems to analyze large volumes of email data and identify patterns that may not be visible through traditional approaches. Machine learning models can learn from past examples and make predictions based on similarities, allowing them to detect new types of spam or phishing emails more effectively. NLP techniques further enhance this process by understanding the context and meaning of the text within emails, rather than just scanning for specific keywords. Additionally, URL-based detection systems are used to examine links embedded in emails. These systems check whether a link leads to a suspicious or previously reported malicious website, helping prevent users from visiting harmful pages.

Despite these technological advancements, existing email security solutions still face several challenges. One of the major drawbacks is their primary focus on classification rather than user awareness. Most systems simply categorize emails as spam or safe without explaining the reasons behind their decisions. This lack of transparency makes it difficult for users to understand the risks associated with certain emails. As a result, users may ignore warnings or fail to learn how to identify threats on their own, which reduces the overall effectiveness of these systems.

Another important limitation is the lack of adaptability in many existing solutions. Cyber threats are constantly evolving, with attackers developing new techniques to bypass security measures. However, some systems are not flexible enough to quickly adapt to these changes. They may require frequent updates or retraining, which can be time-consuming and resource-intensive. Furthermore, advanced security systems that use complex algorithms often demand high computational power and infrastructure. This makes them less suitable for small organizations or real-time applications where quick responses are required.

In addition, many current systems operate as fully automated tools, leaving users dependent on their decisions without any meaningful interaction or feedback. While automation improves efficiency, it

also creates a gap between the system and the user. Without proper guidance or explanation, users remain unaware of how threats are detected and how they can protect themselves. This highlights the need for a more balanced approach that combines intelligent detection with user education.

Overall, while existing email security solutions have made significant progress in detecting and filtering threats, they still lack important features such as transparency, adaptability, and user engagement. Addressing these gaps is essential for building a more effective and user-friendly email security system.

2.4 Evaluation of the Existing Solution

Although existing email security systems provide a basic level of protection, their overall effectiveness is limited in several important aspects. One of the most significant drawbacks is the lack of explainability. Most systems simply label an email as “spam” or “safe” without giving any clear reason for their decision. This lack of transparency reduces user trust and makes it difficult for individuals to understand what exactly made an email suspicious. When users are not informed about the reasoning behind a classification, they miss the opportunity to learn and improve their own awareness, which is essential in preventing future attacks.

Another major limitation is the inability of many systems to detect advanced and modern threats. Cybercriminals today use highly sophisticated techniques, including personalized phishing emails, social engineering tactics, and carefully crafted messages that closely resemble legitimate communication. These emails often avoid obvious spam indicators, making them difficult for traditional filtering systems to detect. As a result, many harmful emails manage to bypass security layers and reach users’ inboxes, increasing the risk of cyber attacks.

The static nature of rule-based systems further weakens their effectiveness. These systems rely on predefined rules and patterns, which means they can only detect threats that match known criteria. However, cyber threats are constantly evolving, with attackers introducing new strategies and variations. Static systems struggle to keep up with these rapid changes, as updating rules frequently can be time-consuming and inefficient. This lack of adaptability creates a gap between existing security measures and the dynamic nature of modern cyber threats.

High false rates are another critical issue in current email security solutions. False positives occur when legitimate emails are incorrectly marked as spam, which can lead to important messages being missed or ignored. On the other hand, false negatives happen when malicious emails are not detected and are allowed to reach the user. Both situations can have serious consequences, either by disrupting communication or by exposing users to potential threats. Achieving a balance between sensitivity and accuracy remains a challenge for many systems.

In addition to these issues, advanced detection models often require significant computational resources. Machine learning-based systems, especially those trained on large datasets, may need

powerful hardware and longer processing times. This can make them less practical for real-time applications or for smaller organizations that do not have access to high-end infrastructure. The need for efficiency and scalability is therefore an important factor that current systems often fail to address.

Perhaps one of the most overlooked limitations is the lack of focus on user awareness. Existing systems are primarily designed to filter emails automatically, with minimal interaction or guidance provided to the user. Even when a threat is detected, users are not given enough information about how to respond or avoid similar risks in the future. This leaves them vulnerable, as they continue to rely entirely on automated systems without developing their own ability to threats.

These limitations clearly indicate that current email security solutions are not fully equipped to handle the evolving and complex nature of cyber threats. There is a strong need for a more intelligent, adaptive, and user-centric approach that not only detects threats effectively but also educates and empowers users to make safer decisions.

2.5 Description of other possible Solutions

To overcome the limitations of existing email security systems, it is important to explore more advanced and intelligent approaches that can handle modern cyber threats effectively. One of the key solutions is the use of advanced machine learning models such as Random Forest, Support Vector Machines, and Gradient Boosting. These algorithms are capable of analyzing complex datasets and identifying hidden patterns that traditional methods often miss. By learning from historical email data, these models can make more accurate predictions and significantly improve the overall classification performance. They also help in reducing false positives and false negatives, which are common issues in basic filtering systems.

In addition to traditional machine learning techniques, deep learning approaches offer even greater potential for improving email threat detection. Models such as Long Short-Term Memory (LSTM) networks and transformer-based architectures are designed to understand sequential and contextual information in text. This makes them highly effective in analyzing email content, as they can capture the meaning, tone, and intent behind messages. For example, a phishing email that uses polite language but contains hidden malicious intent can be better detected using deep learning models. These approaches allow the system to move beyond simple keyword matching and focus on deeper contextual understanding.

Natural Language Processing (NLP) also plays a crucial role in enhancing email security systems. NLP techniques enable the system to process and interpret human language more effectively. By analyzing sentence structure, semantics, and linguistic patterns, NLP helps in identifying suspicious content that may not be obvious at first glance. It can detect unusual phrasing, emotional triggers, or

manipulative language often used in phishing attacks. This adds an extra layer of intelligence to the system, making it more capable of identifying subtle threats.

Another important approach is URL and domain analysis. Many malicious emails include links that redirect users to harmful websites. By analyzing factors such as domain age, URL structure, and reputation, the system can determine whether a link is safe or suspicious. For instance, newly created domains or URLs with unusual patterns are often indicators of phishing attempts. Integrating URL analysis into the detection system helps in preventing users from accessing dangerous websites, thereby reducing the risk of data theft and malware infections.

Hybrid systems are also gaining importance as they combine the strengths of multiple approaches. By integrating rule-based methods, machine learning models, and NLP techniques, hybrid systems can provide better accuracy and flexibility. Rule-based methods can handle known threats efficiently, while machine learning and NLP can detect new and evolving attacks. This combination ensures a more balanced and reliable system that can adapt to different types of threats.

Cloud-based solutions further enhance the effectiveness of email security systems by enabling real-time detection and scalability. These systems can process large volumes of data quickly and update threat information across multiple platforms simultaneously. This is particularly useful for organizations with large user bases, as it ensures consistent protection without requiring heavy local infrastructure.

Finally, incorporating user awareness systems is essential for long-term security. Providing alerts, explanations, and basic training modules helps users understand potential risks and make informed decisions. Instead of relying solely on automated systems, users become active participants in maintaining security. This combination of intelligent detection and user education creates a more robust and effective defense against cyber threats.

Overall, integrating these advanced approaches can lead to the development of a more efficient, adaptable, and user-friendly email threat detection system.

2.6 Evaluation of the possible Solution

The proposed system offers a comprehensive and modern solution to email security by combining intelligent detection mechanisms with strong user awareness features. Unlike traditional systems that rely only on basic filtering, this approach uses machine learning algorithms to analyze multiple aspects of an email, including its subject, content, sender information, and embedded links. By examining these elements together, the system can make more accurate and reliable classifications. The use of preprocessing techniques such as text cleaning, stop-word removal, and feature extraction ensures that the input data is well-structured and relevant, which directly contributes to improved model performance and efficiency.

One of the key strengths of this system is its explainable nature. In many existing solutions, users are simply informed whether an email is spam or safe, without any additional context. However, this system goes a step further by clearly explaining the reasons behind its decisions. For example, it may highlight suspicious keywords, unusual sender details, or potentially harmful links that influenced the classification. This level of transparency not only builds user trust but also helps individuals understand potential risks. Over time, users can develop better judgment and become more cautious while handling emails, making the system both a protective and educational tool.

Another important advantage is the system's user-friendly design. It is developed with simplicity in mind so that even users with limited technical knowledge can easily interact with it. The interface allows users to input email details and receive quick results along with explanations and recommendations. This makes the system highly practical for everyday use. Additionally, the lightweight nature of the system ensures that it can operate efficiently without requiring high computational resources, making it suitable for real-time applications and small-scale environments.

The modular design of the system further enhances its flexibility and scalability. Different components, such as data preprocessing, model training, and prediction, are organized into separate modules. This structure allows developers to update or improve specific parts of the system without affecting the entire application. For instance, new datasets or updated threat patterns can be easily integrated to keep the system up to date with evolving cyber threats. This adaptability ensures that the system remains effective over time.

The integration of a web-based interface adds another layer of convenience. Users can access the system from different devices without the need for complex installations. The interface provides instant feedback, enabling users to quickly determine whether an email is safe or potentially harmful. This real-time interaction is especially useful in preventing users from making risky decisions, such as clicking on malicious links.

Despite these advantages, the system may still face certain challenges. Detecting highly sophisticated or zero-day attacks those that have not been previously identified remains difficult for most machine learning-based systems. Since these attacks do not follow known patterns, they may bypass detection mechanisms. However, this limitation can be addressed over time by continuously updating the model with new data and incorporating advanced techniques such as deep learning and real-time threat intelligence.

Overall, the proposed system represents a significant improvement over existing solutions by combining accuracy, transparency, usability, and adaptability. It not only strengthens email security but also empowers users to make safer and more informed decisions.

Another important aspect of the proposed system is its ability to support continuous learning and improvement. As new types of email threats emerge, the system can be updated with fresh datasets

and retrained models to enhance its detection capabilities. This ensures that the system does not become outdated over time and remains effective against evolving attack patterns. Incorporating feedback mechanisms, such as allowing users to report misclassified emails, can further improve the system's accuracy. This collaborative approach helps in building a more dynamic and responsive security solution.

Furthermore, the system has the potential to be integrated with existing email platforms and organizational infrastructures. By acting as an additional security layer, it can complement current filtering mechanisms rather than replacing them entirely. This makes it easier to adopt in real-world scenarios without requiring major changes to existing systems. In the future, the system can also be expanded to include features such as multilingual email analysis, attachment scanning, and behavioural analysis of senders. These enhancements would make the solution even more robust and capable of handling a wider range of cyber threats in diverse environments.

3. DESIGN

3.1 Project Plan

The project is developed in a well-structured and systematic manner to ensure the smooth design and implementation of the email threat detection system. The entire process is carefully planned and divided into multiple stages, starting from understanding the problem to the final deployment of the system. In the initial phase, the requirements are clearly identified, focusing on the need for accurate email classification and improved user awareness. This stage helps in defining the overall scope, objectives, and functionality of the system.

Following this, the data collection phase is carried out, which is a crucial part of the project. In this stage, relevant datasets containing different types of emails such as safe, spam, and phishing are gathered from reliable sources. The quality and diversity of the dataset play an important role in determining the performance of the system. After collecting the data, the next step involves preprocessing and feature extraction. During preprocessing, the raw email data is cleaned by removing unnecessary elements such as stop words, special characters, and irrelevant information. Feature extraction techniques are then applied to convert the cleaned data into a structured format that can be effectively used by machine learning models.

Once the data is prepared, various classification algorithms are selected and trained using the dataset. These models are evaluated based on their accuracy and efficiency, and the best-performing model is chosen for implementation. After achieving satisfactory results, the model is integrated into a web-based interface, allowing users to interact with the system easily. In the final stage, the system undergoes testing and evaluation using performance metrics such as accuracy, precision, and recall. Based on the results, necessary improvements are made to enhance the system's reliability. This structured approach ensures the successful completion of the project within the planned timeline.

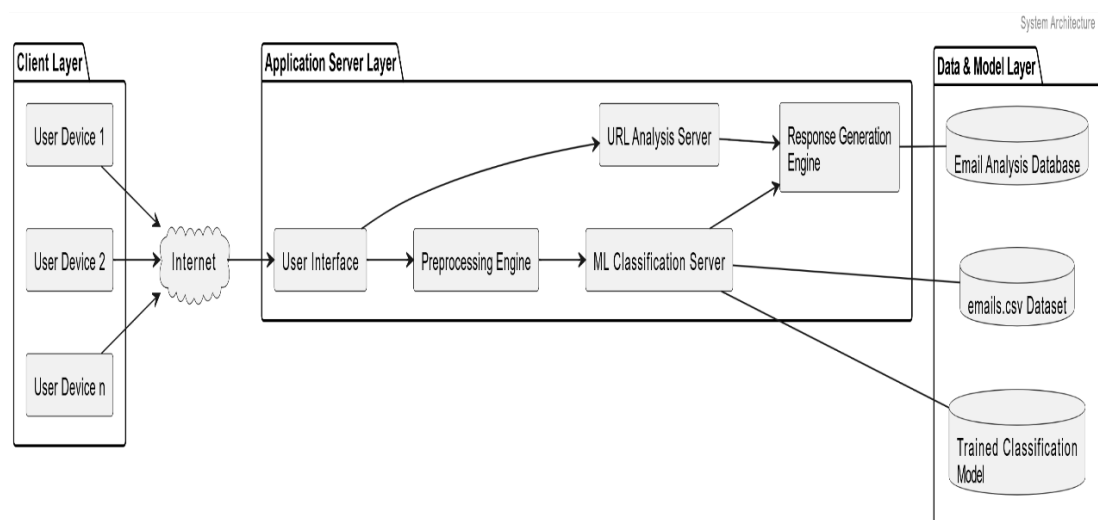
3.2 Description of Method of Solution

The proposed system adopts a machine learning-based approach to effectively detect and classify email threats. The process begins with the collection of email data, which includes essential components such as the subject line, body content, and embedded URLs. These elements provide valuable information that helps in identifying whether an email is safe or potentially harmful. Collecting diverse and well-structured data is an important step, as it directly influences the performance and accuracy of the system.

Once the data is collected, it undergoes preprocessing to improve its quality and usability. In this stage, unnecessary characters, special symbols, and formatting inconsistencies are removed. Stop words commonly used words that do not add significant meaning—are also eliminated to focus on the most relevant information. This cleaning process ensures that the data is more structured and suitable for analysis, reducing noise and improving the efficiency of the model. After preprocessing, feature extraction techniques such as TF-IDF (Term Frequency-Inverse Document Frequency) are applied to convert textual data into numerical form. Since machine learning models cannot directly process text, this step is essential for transforming the data into a format that the algorithms can understand. TF-IDF helps in identifying the importance of words in an email by considering both their frequency and uniqueness across the dataset. This allows the system to capture meaningful patterns and relationships within the content. Following feature extraction, machine learning algorithms such as Logistic Regression and Naïve Bayes are used to classify emails into categories like Safe, Spam, or Phishing. These models are trained using labelled data so that they can learn to distinguish between different types of emails based on their features. Once trained, they can make accurate predictions on new, unseen emails.

The system also includes a prediction module that allows users to input new email content and receive instant classification results. In addition to detection, an insight module is integrated to provide explanations for the classification. It highlights suspicious elements and offers safety tips, helping users understand the reasoning behind the result. This combination of intelligent detection and user guidance makes the system both effective and easy to use.

3.3 System Design and System Architecture



The given sequence diagram provides a clear and detailed representation of how the email threat detection system operates by showing the interaction between different components in a step-by-step manner. It begins with the **user**, who plays a central role in initiating the process. The user enters the email subject and body through the **web interface**, which acts as a bridge between the user and the backend system. This interface is designed to be simple and user-friendly so that even individuals with limited technical knowledge can easily input email content and receive results.

Once the user submits the email details, the web interface sends the raw email content to the **preprocessing module**. This stage is very important because the data collected from emails is often unstructured and may contain unnecessary characters, formatting issues, or irrelevant words. The preprocessing module cleans the data by removing stopwords, special symbols, and noise, and converts it into a structured format that can be easily analyzed. This ensures that only meaningful and relevant information is passed forward, which improves the overall performance of the system.

After preprocessing, the cleaned text is sent to the **classification service**, which is the core component of the system. This module uses machine learning algorithms to analyze the email content and identify patterns that indicate whether the email is safe or spam (or potentially phishing). Based on the trained model, the system generates a classification result. This step is crucial because it determines the nature of the email using learned knowledge from previous data, making the detection process more accurate and intelligent compared to traditional rule-based systems.

Once the classification result is obtained, it is sent back within the system and forwarded to the **response generator**. This module enhances the system by adding an extra layer of user awareness. Instead of just displaying a simple result, the response generator creates meaningful insights and suggestions. For example, it may highlight suspicious words, unusual links, or unknown sender details that contributed to the classification. It also provides safety tips, such as avoiding clicking on unknown links or verifying the sender's identity before taking any action. This feature helps users understand the reasoning behind the result and improves their ability to identify threats in the future.

Finally, the web interface receives both the classification result and the generated insights, and displays the final output to the user. This output includes the email category (safe or spam), along with explanations and recommended actions. The entire process happens in a smooth and sequential flow, ensuring quick and efficient results.

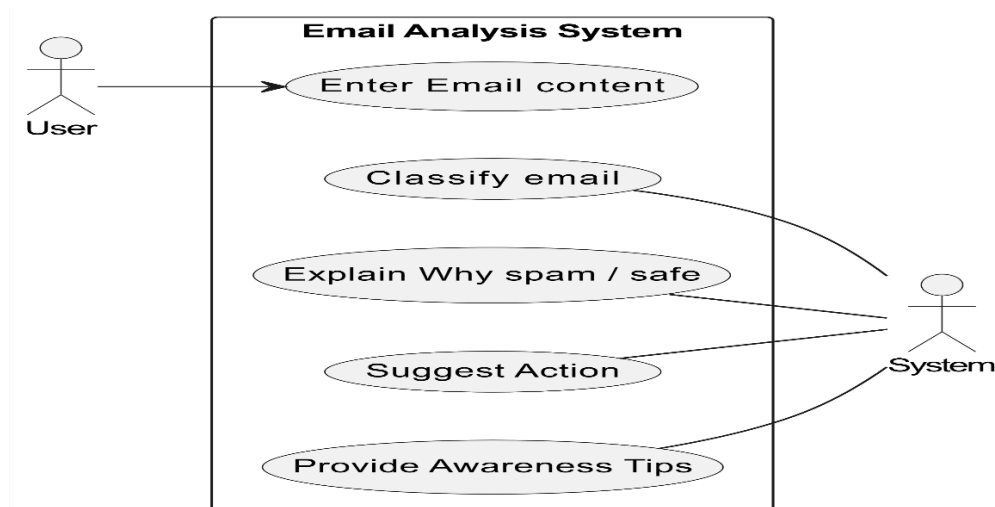
Overall, this sequence diagram highlights how different modules—user interface, preprocessing, classification, and response generation—work together in a coordinated manner. It not only shows the technical workflow but also emphasizes the importance of user interaction and awareness. By combining automation with clear explanations, the system provides a balanced solution that is both effective and user-friendly, ultimately helping users make safer decisions while handling emails.

3.4 UML Diagrams

UML (Unified Modelling Language) diagrams play an important role in designing and visualizing the structure and behaviour of the proposed email threat detection system. They help in clearly representing how different components of the system interact with each other and how the overall process flows. For example, a use case diagram shows the interaction between the user and the system, including actions like inputting email data and receiving classification results. Similarly, an activity diagram explains the step-by-step workflow, from email input to preprocessing, classification, and final output.

In addition, class diagrams and sequence diagrams are used to represent the internal structure and communication within the system. A class diagram defines different modules such as data processing, feature extraction, and prediction, along with their relationships. Sequence diagrams illustrate how these modules interact in a specific order to perform a task. Overall, UML diagrams make the system design more organized, easy to understand, and helpful for both development and documentation purposes.

3.4.1 Use Case Diagram

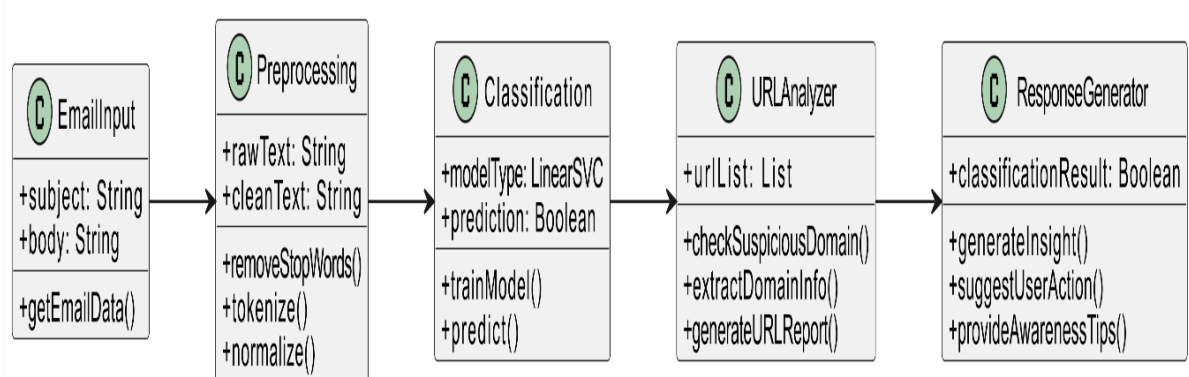


This use case diagram illustrates the overall working of the email threat detection system by showing how the user interacts with different system functionalities. The process begins with the user entering the email content, such as the subject and body. This input acts as the foundation for the entire analysis. The diagram highlights a simple and user-friendly interaction, ensuring that even non-technical users can easily use the system without any complexity.

After receiving the input, the system performs the core function of classifying the email as safe, spam, or potentially harmful. This is done using machine learning techniques that analyze the content and identify patterns. In addition to classification, the system also explains why the email is considered safe or spam. This feature improves transparency by helping users understand the reasoning behind the result, rather than just showing a label.

The system further enhances user support by suggesting appropriate actions and providing awareness tips. For example, it may advise users to avoid clicking on unknown links or to verify sender details. These features not only help in immediate decision-making but also educate users for future situations. Overall, the diagram reflects a balanced approach where the system focuses on both accurate detection and improving user awareness.

3.4.2 Class Diagram



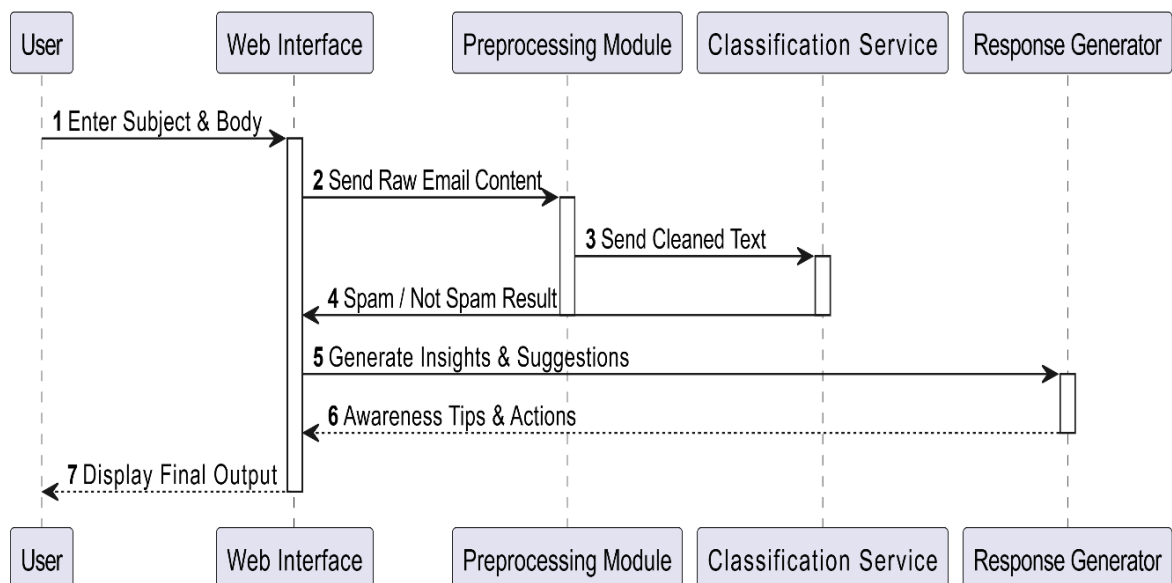
This class diagram represents the internal structure of the email threat detection system by showing different modules and their responsibilities. The process starts with the **Email Input** class, which captures the subject and body of the email from the user. This data is then passed to the **Preprocessing** module, where the raw text is cleaned and transformed into a usable

format. Functions like removing stop words, tokenization, and normalization help in preparing high-quality data for further analysis.

After preprocessing, the cleaned data is sent to the **Classification** module, which acts as the core component of the system. This module uses a machine learning model (such as Linear) to train on the data and make predictions about whether the email is safe or harmful. Alongside this, the **URL Analyzer** module examines any links present in the email. It checks for suspicious domains, extracts relevant domain information, and generates a report to identify potentially malicious URLs, adding an extra layer of security.

Finally, the results from both classification and URL analysis are passed to the **Response Generator** module. This component is responsible for generating the final output, including the classification result, detailed insights, suggested user actions, and awareness tips. It ensures that the user not only receives a decision but also understands the reasoning behind it. Overall, the diagram shows a well-organized and modular system where each component works together to provide accurate detection and meaningful user guidance.

3.4.3 Sequence Diagram



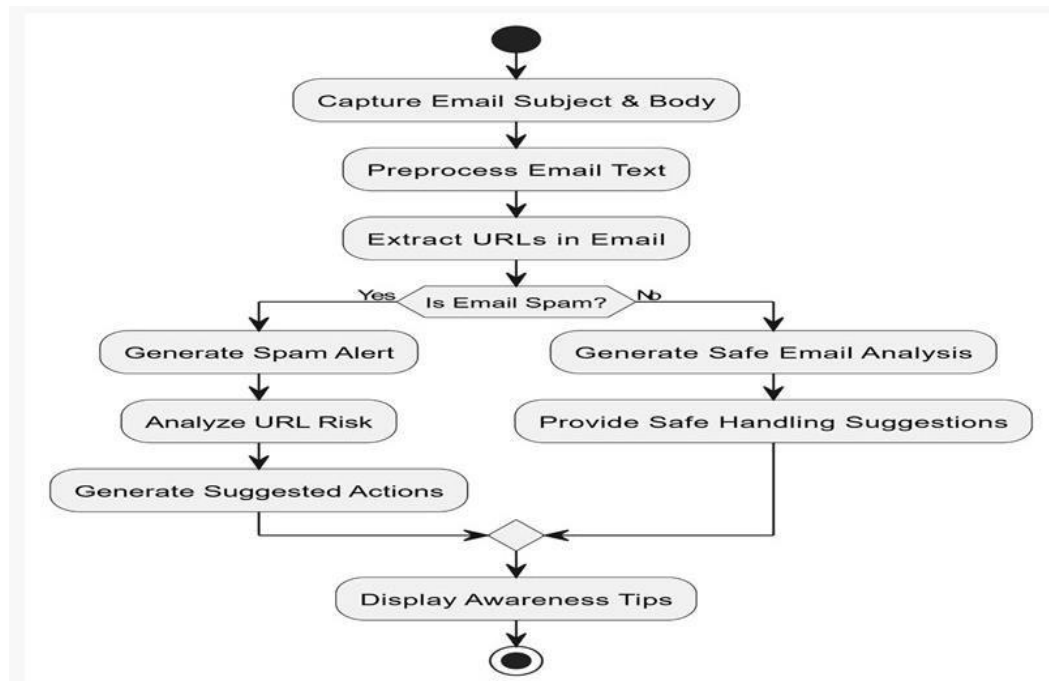
This sequence diagram represents the complete workflow of the email threat detection system by showing how different components interact in a step-by-step manner. The process begins with the **user**, who enters the email subject and body through the **web interface**. This interface acts as a communication layer between the user and the backend system. Once the input is

provided, the web interface forwards the raw email content to the **preprocessing module**, where the data is cleaned and prepared for analysis by removing unnecessary elements and formatting issues.

After preprocessing, the cleaned text is sent to the **classification service**, which is responsible for analysing the email using machine learning techniques. This module evaluates the content and generates a result indicating whether the email is spam or not. The classification result is then sent back through the system. At this stage, the system not only focuses on detection but also moves towards providing meaningful insights. The processed data is forwarded to the **response generator**, which plays an important role in enhancing user understanding.

The **response generator** creates detailed insights, suggestions, and awareness tips based on the classification result. It helps the user understand why an email is considered safe or suspicious and provides recommended actions, such as avoiding unknown links or verifying sender information. Finally, the web interface displays the complete output to the user, including the classification result and guidance. Overall, the diagram shows a smooth and logical flow where each module works together to ensure accurate detection and improved user awareness

3.4.4 Activity Diagram



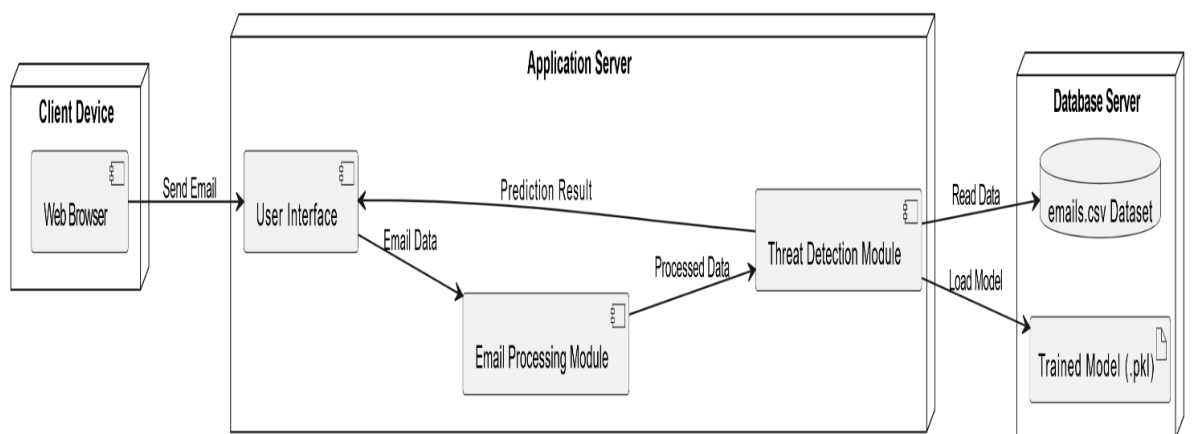
This activity diagram represents the step-by-step workflow of the email threat detection system, clearly showing how the process flows from start to end. The process begins with capturing the

email subject and body from the user. This input is then passed to the preprocessing stage, where the email text is cleaned and prepared for analysis. After preprocessing, the system detects any URLs present in the email, as links often play a key role in identifying phishing or malicious content.

The next step involves a decision-making process where the system determines whether the email is spam or not. If the email is identified as spam, the system generates a spam alert and proceeds to analyse the risk associated with any URLs found in the email. Based on this analysis, it generates suggested actions for the user, such as avoiding links or deleting the email. On the other hand, if the email is considered safe, the system generates safe email insights and provides recommendations for secure handling, ensuring the user understands that the email does not pose a threat.

Finally, regardless of the classification result, the system displays awareness tips to guide the user in making better decisions while handling emails in the future. These tips act as an educational component, helping users recognize potential threats independently. The process then ends after presenting all the necessary information. Overall, this diagram highlights a logical and structured flow that combines detection, decision-making, and user guidance to enhance email security.

3.4.5 Deployment Diagram



The deployment diagram illustrates the overall architecture and interaction between different components of the Email Threat Detection System. It provides a clear understanding of how data flows from the user to the system and how the system processes that data to generate

meaningful results. This diagram mainly consists of three major parts: the Client Device, the Application Server, and the Database Server.

The process begins at the Client Device, where the user interacts with the system through a web browser. The web browser acts as the front-end platform, allowing users to input or send email data that needs to be analysed. This could be in the form of a complete email message or specific content that the user wants to verify. Once the user submits the email, the data is transmitted to the Application Server for further processing.

The Application Server is the central component of the system where all major operations take place. It contains multiple modules that work together to analyse the email data. The first module within the server is the User Interface (UI). This interface not only receives the input data from the client but also displays the final prediction results back to the user. It ensures smooth communication between the user and the backend system.

After receiving the email data, the UI forwards it to the Email Processing Module. This module plays a crucial role in preparing the data for analysis. Since raw email data is usually unstructured and may contain noise such as special characters, unnecessary words, or irrelevant information, preprocessing is essential. The module performs several tasks including data cleaning, tokenization, normalization, removal of stop words, and feature extraction. These steps convert the raw email into a structured format that can be understood by machine learning algorithms.

Once the preprocessing is completed, the refined data is passed to the Threat Detection Module. This is the core component of the system, where the actual classification takes place. The module uses a trained machine learning model to analyse the processed data and determine whether the email is safe or potentially harmful. The model is typically stored in a serialized format and is loaded during runtime.

The Database Server supports the system by storing important resources such as datasets and trained models. The dataset is used during the training phase of the model, while the trained model file is used during prediction. The Threat Detection Module may access the database to retrieve the model or reference data when required.

Finally, once the analysis is completed, the prediction result (such as “Spam” or “Not Spam”) is sent back to the User Interface, which displays it to the user. This completes the process.

Overall, the deployment diagram demonstrates a well-structured and efficient system design that ensures accurate, fast, and reliable email threat detection through seamless interaction between all components.

3.5 Hardware Requirements

The email threat detection system is designed to be simple, efficient, and accessible, which means it does not require high-end or expensive hardware to function effectively. One of the main goals of this project is to ensure that it can be easily used by students, developers, and small organizations without the need for specialized infrastructure. The system is lightweight in nature and can run smoothly on a standard computing device such as a personal laptop or desktop.

A basic **processor such as Intel i3 or higher** is sufficient to run the system. Since the project involves machine learning operations like data preprocessing and classification, having a moderately capable processor helps in executing tasks efficiently. However, the system is optimized in such a way that it does not demand high computational power, making it suitable for everyday devices.

In terms of memory, a **minimum of 4 GB RAM** is required for the system to function properly. This allows the system to handle data processing and model execution without major delays. For better performance, especially when working with larger datasets or running multiple processes simultaneously, **8 GB RAM is recommended**. This ensures smoother execution and reduces the chances of lag during real-time predictions.

The system also requires **at least 10 GB of free storage space**. This space is necessary to store datasets, trained models, application files, and other related resources. Having sufficient storage ensures that the system can operate without interruptions and allows room for future updates or expansion, such as adding more data or improving the model.

An **internet connection** is another important requirement, especially during the development and deployment phases. It is needed to download datasets, install necessary libraries, and access cloud-based resources if required. Additionally, internet connectivity can be useful for integrating the system with web-based platforms or updating threat information in real time.

Basic **input devices such as a keyboard and mouse** are required for user interaction, allowing users to enter email content and navigate through the system interface. Similarly, an **output device like a monitor** is essential to display the classification results, explanations, and awareness tips clearly to the user.

Overall, these hardware requirements highlight the practicality and accessibility of the system. By keeping the requirements minimal, the project ensures that it can be widely adopted without financial burden, making advanced email security solutions available to a larger audience.

3.6 Software Requirements

The email threat detection system is built using modern and widely used software tools that support both machine learning and web-based application development. The primary programming language used for this project is **Python**, which is known for its simplicity, flexibility, and strong support for data science and artificial intelligence. Python provides a wide range of libraries and frameworks that make it easier to implement complex functionalities such as data processing, model training, and prediction.

For development and testing, environments like **Jupyter Notebook**, **Visual Studio Code (VS Code)**, or **PyCharm** can be used. Jupyter Notebook is especially useful during the initial stages for experimenting with datasets and testing machine learning models, as it allows step-by-step execution and easy visualization. On the other hand, VS Code and PyCharm provide a more structured environment for writing, debugging, and managing the complete application, making them suitable for final development and integration.

The system makes use of several important Python libraries to perform different tasks efficiently. **Scikit-learn** is used for implementing machine learning algorithms such as classification models, which play a key role in detecting whether an email is safe or harmful. **Pandas and NumPy** are used for data handling and numerical operations, helping in organizing and processing large amounts of email data. For text processing, libraries like **NLTK or spacey** are used to clean, tokenize, and analyse the email content, enabling better understanding of the text.

To create a user-friendly interface, web frameworks such as **Flask** are used. These frameworks allow the system to be deployed as a web application, where users can easily input email content and receive results through a browser. Flask is lightweight and simple, making it ideal for smaller applications, while Django offers more advanced features for larger and scalable systems.

A **lightweight database such as SQLite** is used to store necessary data, including email records, model outputs, or user inputs if required. It is easy to set up and does not require complex configuration, making it suitable for this project.

The system is compatible with multiple operating systems, including **Windows, macOS, and Linux**, ensuring flexibility and ease of use. Overall, these software requirements support efficient development, smooth testing, and reliable deployment of the system.

4. IMPLEMENTATION

Sample Code :

HTML Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Intelligent Email Threat Detector</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;800&display=swap
" rel="stylesheet">
</head>
<body>
  <div class="background-effects">
    <div class="glow glow-1"></div>
    <div class="glow glow-2"></div>
  </div>

  <!-- Center Module -->
  <main class="center-module">
    <h1 class="welcome-title">Intelligent Email Threat Detection</h1>
    <p class="subtitle">Empowering you with AI-driven analysis to stay safe from evolving
threats.</p>
    <button id="btn-open-analyzer" class="btn-primary">Email Analyzer</button>

    <!-- Results Container -->
    <div id="results-container" class="hidden">
```



```

<h2 id="res-classification"></h2>
  <div class="insight-box">
    <h3>Email Insight</h3>
    <p id="res-email-insight"></p>
  </div>
  <div class="insight-box hidden" id="box-url-insight">
    <h3>URL Insight</h3>
    <p id="res-url-insight"></p>
  </div>
  <div class="insight-box action-box">
    <h3>Suggested Action</h3>
    <p id="res-action"></p>
  </div>
  <div class="insight-box awareness-box">
    <h3>Awareness Tips</h3>
    <p id="res-awareness"></p>
  </div>
  <button id="btn-close-results" class="btn-secondary mt-2">Close Analysis</button>
</div>
</main>

```

```

<!-- Corner Modules -->
<!-- Left Corner Container -->
<div class="left-corner-container">
  <div class="corner-module" id="mod-confidence">
    <div class="widget-icon"><img alt="Bar chart icon" data-bbox="422 708 445 725"/></div>
    <span>Confidence Score</span>
  </div>

  <div class="corner-module" id="mod-dashboard">
    <div class="widget-icon"><img alt="Checkmark icon" data-bbox="422 832 445 849"/></div>
    <span>Threat Dashboard</span>
  </div>

```

```
<div class="corner-module" id="mod-quiz">
```

```
  <div class="widget-icon">□</div>
```

```
  <span>Awareness Quiz</span>
```

```
</div>
```

```
<div class="corner-module" id="mod-summary">
```

```
  <div class="widget-icon">☰</div>
```

```
  <span>Email Summary</span>
```

```
</div>
```

```
<!-- <div class="corner-module" id="mod-feedback">
```

```
  <div class="widget-icon">💬</div>
```

```
  <span>Feedback</span>
```

```
</div> -->
```

```
</div>
```

```
<!-- Modals -->
```

```
<!-- Analyzer Modal -->
```

```
<div id="modal-analyzer" class="modal hidden">
```

```
  <div class="modal-content glass">
```

```
    <button class="close-btn" onclick="closeModal('modal-analyzer')">&times;</button>
```

```
    <h2>Analyze Email</h2>
```

```
    <form id="analyzer-form">
```

```
      <input type="text" id="inp-subject" placeholder="Subject" required>
```

```
      <textarea id="inp-body" rows="5" placeholder="Paste email body here..."
required></textarea>
```

```
      <textarea id="inp-urls" rows="2" placeholder="Paste URLs here
(optional)"></textarea>
```

```
      <button type="submit" class="btn-primary" id="btn-submit-analysis">Analyze
Now</button>
```

```
    </form>
```

```
    <div id="loading" class="hidden">Analyzing...</div>
```

```
</div>
```

</div>

<!-- Confidence Modal -->

<div id="modal-confidence" class="modal hidden">

<div class="modal-content glass">

<button class="close-btn" onclick="closeModal('modal-confidence')">×</button>

<h2>Confidence Score</h2>

<div class="score-display">

<h1 id="conf-score">--%</h1>

<h3 id="conf-level">--</h3>

</div>

<div class="info-text">

Meaning:

<p id="conf-meaning">Run an analysis first to see the model's certainty.</p>

Reliability:

<p>The system is highly reliable when strong threat indicators are present.</p>

</div>

</div>

</div>

<!-- Dashboard Modal -->

<div id="modal-dashboard" class="modal hidden">

<div class="modal-content glass">

<button class="close-btn" onclick="closeModal('modal-dashboard')">×</button>

<h2>Threat Dashboard</h2>

<div class="dashboard-stats">

<div class="stat-card">

<h4>Total Analyzed</h4>

<h2 id="dash-total">0</h2>

</div>

<div class="stat-card safe">

```

        <h4>Safe</h4>
        <h2 id="dash-safe">0</h2>
    </div>
    <div class="stat-card spam">
        <h4>Spam</h4>
        <h2 id="dash-spam">0</h2>
    </div>
    <div class="stat-card phishing">
        <h4>Phishing</h4>
        <h2 id="dash-phishing">0</h2>
    </div>
</div>
</div>
</div>

```

```

<!-- Quiz Modal -->
<div id="modal-quiz" class="modal hidden">
    <div class="modal-content glass">
        <button class="close-btn" onclick="closeModal('modal-quiz')">&times;</button>
        <h2>☐ Awareness Quiz</h2>
        <div id="quiz-question">
            <p><strong>Question:</strong> An email asks you to verify your bank account
urgently. What should you do?</p>
            <div class="quiz-options">
                <button class="quiz-btn" onclick="checkAnswer(false)">(A) Click
immediately</button>
                <button class="quiz-btn" onclick="checkAnswer(false)">(B) Ignore warning
signs</button>
                <button class="quiz-btn" onclick="checkAnswer(true)">(C) Verify sender before
action</button>
            </div>
        </div>
        <div id="quiz-result" class="hidden">

```

```

        <h3 id="quiz-feedback"></h3>
        <p><strong>Explanation:</strong> Phishing emails create urgency to trick users.
Always verify the sender before clicking links.</p>
        <button class="btn-secondary" onclick="resetQuiz()">Try Another</button>
    </div>
</div>
</div>

<!-- Summary Modal -->
<div id="modal-summary" class="modal hidden">
    <div class="modal-content glass">
        <button class="close-btn" onclick="closeModal('modal-summary')">&times;</button>
        <h2>📧 Email Summary</h2>
        <div class="info-text">
            <p id="sum-text">Run an analysis first to generate a summary.</p>
            <hr>
            <strong>Key Intent:</strong>
            <p id="sum-intent">Waiting for analysis...</p>
        </div>
    </div>
</div>

<!-- Feedback Modal -->
<div id="modal-feedback" class="modal hidden">
    <div class="modal-content glass">
        <button class="close-btn" onclick="closeModal('modal-feedback')">&times;</button>
        <h2>Feedback</h2>
        <p>Help us improve our threat detection.</p>
        <textarea rows="4" placeholder="Your feedback here..." style="width: 100%; box-
sizing: border-box;"></textarea>
        <button class="btn-primary mt-2" onclick="closeModal('modal-
feedback')">Submit</button>
    </div>

```

</div>

<script src="{{ url_for('static', filename='js/script.js') }}"></script>

</body>

</html>

Script.js

```
const btnOpenAnalyzer = document.getElementById('btn-open-analyzer');
const modalAnalyzer = document.getElementById('modal-analyzer');
const btnCloseResults = document.getElementById('btn-close-results');
const resultsContainer = document.getElementById('results-container');
const formAnalyzer = document.getElementById('analyzer-form');
const loading = document.getElementById('loading');
```

// Modules

```
const modules = ['confidence', 'dashboard', 'quiz', 'summary', 'feedback'];
modules.forEach(mod => {
  const el = document.getElementById(`mod-${mod}`);
  if (el) {
    el.addEventListener('click', () => {
      openModal(`modal-${mod}`);
      if(mod === 'dashboard') fetchDashboard();
    });
  }
});
```

```
function openModal(id) {
  document.getElementById(id).classList.remove('hidden');
}
```

```
function closeModal(id) {
  document.getElementById(id).classList.add('hidden');
```

```

}

btnOpenAnalyzer.addEventListener('click', () => {
  openModal('modal-analyzer');
});

btnCloseResults.addEventListener('click', () => {
  resultsContainer.classList.add('hidden');
  btnOpenAnalyzer.classList.remove('hidden');
  document.querySelector('.welcome-title').classList.remove('hidden');
  document.querySelector('.subtitle').classList.remove('hidden');
});

formAnalyzer.addEventListener('submit', async (e) => {
  e.preventDefault();
  loading.classList.remove('hidden');
  formAnalyzer.classList.add('hidden');

  const payload = {
    subject: document.getElementById('inp-subject').value,
    body: document.getElementById('inp-body').value,
    urls: document.getElementById('inp-urls').value
  };

  try {
    const response = await fetch('/analyze', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    const result = await response.json();

    displayResults(result, payload.urls);
  }

```

```

        updateBackgroundState(result);

        closeModal('modal-analyzer');
    } catch (err) {
        console.error(err);
        alert('Analysis failed.');
```

```

    } finally {
        loading.classList.add('hidden');
        formAnalyzer.classList.remove('hidden');
        formAnalyzer.reset();
    }
});

function displayResults(data, urls) {
    document.querySelector('.welcome-title').classList.add('hidden');
    document.querySelector('.subtitle').classList.add('hidden');
    btnOpenAnalyzer.classList.add('hidden');
    resultsContainer.classList.remove('hidden');

    const cl = document.getElementById('res-classification');
    cl.innerText = `Classification: ${data.classification} Email`;
    cl.style.color = getColor(data.classification);

    document.getElementById('res-email-insight').innerText = data.email_insight;
    document.getElementById('res-action').innerText = data.suggested_action;
    document.getElementById('res-awareness').innerText = data.awareness_tips;

    const urlBox = document.getElementById('box-url-insight');
    // Case 1 vs Case 2
    if (urls && urls.trim().length > 0 && data.url_insight) {
        document.getElementById('res-url-insight').innerText = data.url_insight;
        urlBox.classList.remove('hidden');
    } else {

```



```

        urlBox.classList.add('hidden');
    }

    // Update global state for modals
    updateModalsWithData(data);
}

function updateModalsWithData(data) {
    // Confidence
    document.getElementById('conf-score').innerText = data.confidence;
    document.getElementById('conf-level').innerText = data.confidence_level;
    document.getElementById('conf-meaning').innerText = `The model detected patterns
indicative of ${data.classification.toLowerCase()} emails.`;

    // Summary
    document.getElementById('sum-text').innerText = data.summary;
    document.getElementById('sum-intent').innerText = data.email_insight;
}

async function fetchDashboard() {
    try {
        const res = await fetch('/dashboard');
        const data = await res.json();
        document.getElementById('dash-total').innerText = data.total;
        document.getElementById('dash-safe').innerText = data.counts.Safe;
        document.getElementById('dash-spam').innerText = data.counts.Spam;
        document.getElementById('dash-phishing').innerText = data.counts.Phishing;
    } catch(err) { console.error(err); }
}

function getColor(classification) {
    if(classification === 'Phishing') return '#ef4444';
    if(classification === 'Spam') return '#f59e0b';

```

```

    return '#10b981';
}

function updateBackgroundState(data) {
    const col = getColor(data.classification);
    document.querySelector('.glow-1').style.background = `radial-gradient(circle, ${col} 0%,
transparent 70%)`;
}

// Quiz actions
function checkAnswer(isCorrect) {
    document.getElementById('quiz-question').classList.add('hidden');
    const res = document.getElementById('quiz-result');
    res.classList.remove('hidden');
    const fb = document.getElementById('quiz-feedback');
    if(isCorrect) {
        fb.innerText = "✅ Correct!";
        fb.style.color = "#10b981";
    } else {
        fb.innerText = "❌ Incorrect.";
        fb.style.color = "#ef4444";
    }
}

function resetQuiz() {
    document.getElementById('quiz-question').classList.remove('hidden');
    document.getElementById('quiz-result').classList.add('hidden');
}

```

Style.css

```

:root {
    --bg-color: #050505;

```

```

--text-color: #ffffff;
--primary: #6366f1;
--primary-hover: #4f46e5;
--secondary: #333333;
--glass-bg: rgba(255, 255, 255, 0.08);
--glass-border: rgba(255, 255, 255, 0.12);

--phishing: #ef4444;
--spam: #f59e0b;
--safe: #10b981;
}

html, body {
  margin: 0;
  padding: 0;
  font-family: 'Inter', sans-serif;
  color: var(--text-color);
  height: 100vh;
  overflow: hidden;
  position: relative;
  background: url("../images/bg.png") no-repeat center center fixed;
  background-size: cover;
}

/* stronger dark overlay */
body::before {
  content: "";
  position: fixed;
  inset: 0;
  background: rgba(0, 0, 0, 0.82);
  z-index: 0;
  pointer-events: none;
}

```

```
.background-effects {  
  position: fixed;  
  inset: 0;  
  z-index: 1;  
  overflow: hidden;  
  pointer-events: none;  
}
```

```
.glow {  
  position: absolute;  
  width: 500px;  
  height: 500px;  
  background: radial-gradient(circle, var(--primary) 0%, transparent 70%);  
  opacity: 0.12;  
  filter: blur(80px);  
  border-radius: 50%;  
  animation: float 10s infinite alternate;  
}
```

```
.glow-1 { top: -10%; left: -10%; }
```

```
.glow-2 {  
  bottom: -10%;  
  right: -10%;  
  background: radial-gradient(circle, #8b5cf6 0%, transparent 70%);  
  animation-delay: -5s;  
}
```

```
@keyframes float {  
  0% { transform: translateY(0) scale(1); }  
  100% { transform: translateY(30px) scale(1.1); }  
}
```

```

gap: 10px;
  cursor: pointer;
  transition: all 0.3s ease;
  z-index: 5;
  font-weight: 600;
  user-select: none;
}

.corner-module:hover {
  background: rgba(255, 255, 255, 0.1);
  transform: scale(1.05);
}

.widget-icon { font-size: 1.5rem; }

.top-left { top: 20px; left: 20px; }
.top-right { top: 20px; right: 20px; }
.bottom-left { bottom: 20px; left: 20px; }
.bottom-right { bottom: 20px; right: 20px; }
.bottom-center { bottom: 20px; left: 50%; transform: translateX(-50%); }
.bottom-center:hover { transform: translateX(-50%) scale(1.05); }

/* Modals */
.modal {
  position: fixed;
  top: 0; left: 0; width: 100%; height: 100%;
  background: rgba(0, 0, 0, 0.7);
  backdrop-filter: blur(5px);
  display: flex;
  justify-content: center;
  align-items: center;
  z-index: 100;
}

```

```
.glass {
  background: #111111;
  border: 1px solid var(--glass-border);
  box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
  border-radius: 16px;
  padding: 30px;
  width: 90%;
  max-width: 500px;
  position: relative;
  animation: slideUp 0.3s ease-out;
}
```

```
@keyframes slideUp {
  from { opacity: 0; transform: translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}
```

```
.close-btn {
  position: absolute;
  top: 15px; right: 20px;
  background: none; border: none;
  color: #9ca3af; font-size: 1.5rem;
  cursor: pointer;
}
```

```
.close-btn:hover { color: white; }
```

```
/* Forms */
```

```
input, textarea {
  width: 100%;
  background: rgba(255,255,255,0.05);
  border: 1px solid var(--glass-border);
  color: white;
```

```

padding: 12px;
border-radius: 8px;
margin-bottom: 15px;
font-family: inherit;
box-sizing: border-box;
font-size: 1rem;
}
input:focus, textarea:focus {
    outline: none;
    border-color: var(--primary);
}

/* Results Area */
#results-container {
    background: var(--glass-bg);
    border: 1px solid var(--glass-border);
    border-radius: 16px;
    padding: 20px;
    margin-top: 20px;
    text-align: left;
    backdrop-filter: blur(10px);
}

.insight-box {
    background: rgba(0,0,0,0.3);
    border-radius: 8px;
    padding: 10px 15px;
    margin-bottom: 10px;
    border-left: 4px solid var(--primary);
}

.insight-box h3 { margin: 0 0 5px 0; font-size: 0.9rem; color: #a5b4fc; }
.insight-box p { margin: 0; font-size: 0.95rem; line-height: 1.4; }

```

```
.mt-2 { margin-top: 15px; }
```

```
/* Dashboard */
```

```
.dashboard-stats {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 15px;  
}  
.stat-card {  
  background: rgba(255,255,255,0.05);  
  padding: 15px;  
  border-radius: 10px;  
  text-align: center;  
}  
.stat-card h4 { margin: 0; color: #9ca3af; }  
.stat-card h2 { margin: 10px 0 0 0; font-size: 2rem; }  
.stat-card.safe h2 { color: var(--safe); }  
.stat-card.spam h2 { color: var(--spam); }  
.stat-card.phishing h2 { color: var(--phishing); }
```

```
/* Quiz */
```

```
.quiz-options { display: flex; flex-direction: column; gap: 10px; margin-top: 15px; }  
.quiz-btn {  
  background: rgba(255,255,255,0.05);  
  border: 1px solid var(--glass-border);  
  color: white; padding: 10px; border-radius: 8px;  
  cursor: pointer; text-align: left; transition: background 0.2s;  
}  
.quiz-btn:hover { background: rgba(255,255,255,0.1); }
```

```
/* Score */
```

```
.score-display { text-align: center; margin: 20px 0; }  
.score-display h1 { font-size: 4rem; margin: 0; color: var(--primary); }
```



```
.score-display h3 { margin: 5px 0 20px 0; color: #a5b4fc; }
```

```
/* Left corner container */
```

```
.left-corner-container {  
  position: fixed;  
  top: 200px;  
  left: 20px;  
  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
  
  z-index: 1000;  
}
```

```
/* Remove old positions */
```

```
.corner-module {  
  position: relative; /* override absolute */  
}
```

```
/* Optional: make them look cleaner */
```

```
.corner-module {  
  background: var(--glass-bg);  
  border: 1px solid var(--glass-border);  
  padding: 12px 16px;  
  border-radius: 12px;  
  backdrop-filter: blur(10px);  
  cursor: pointer;  
  transition: 0.3s;  
}
```

```
.corner-module:hover {
```

```
        transform: translateX(5px);
    }
}
```

app.py

```
this from flask import Flask, request, jsonify, render_template
import joblib
import random
import os
```

```
app = Flask(__name__)
model = joblib.load('model.pkl')
```

```
total_analyzed = 0
counts = {"Safe": 0, "Spam": 0, "Phishing": 0}
```

```
@app.route('/')
def index():
    return render_template('index.html')
```

```
@app.route('/analyze', methods=['POST'])
```

```
def analyze():
    global total_analyzed, counts
    data = request.json
```

```
    subject = data.get('subject', "")
    body = data.get('body', "")
    urls = data.get('urls', "")
```

```
    text = subject + " - " + body
```

```
    # Predict
```

```
    prediction = model.predict([text])[0]
```

```

# Dashboard update
total_analyzed += 1
counts[prediction] += 1

# Confidence Score (Randomizing for demo based on prediction strength if possible, or just
mock high confidence)
try:
    proba = model.predict_proba([text])[0]
    confidence = max(proba) * 100
except:
    confidence = random.uniform(88, 98)

confidence_level = "High Confidence" if confidence > 85 else "Moderate Confidence"

# Insights Strategy
if prediction == "Phishing":
    insight = "This email creates urgency and requests sensitive action, a common phishing
tactic."
    action = "Do not click any links. Report and delete immediately."
elif prediction == "Spam":
    insight = "This email appears to be an unsolicited promotion or scam."
    action = "Ignore and move to spam folder. Avoid replying."
else:
    insight = "This email appears normal and lacks typical threat indicators."
    action = "Safe to read, but always remain vigilant."

response = {
    "classification": prediction,
    "email_insight": insight,
    "suggested_action": action,
    "confidence": f"{confidence:.0f}%",
    "confidence_level": confidence_level,

```

```

    "awareness_tips": "Always verify the sender before clicking links or downloading
attachments. Phishing often uses urgency to trick you."
}

```

```

# URL insight (only if URLs provided - Case 2)
if urls and urls.strip() != "":
    if prediction == "Phishing":
        response["url_insight"] = "The link likely redirects to a fake login page designed to steal
credentials."
    elif prediction == "Spam":
        response["url_insight"] = "The link may lead to an unsafe promotional site."
    else:
        response["url_insight"] = "The link appears to match the sender's domain, but proceed
with caution."

```

```

# Summarization
summary_text = "This email "
if "urgent" in text.lower() or "verify" in text.lower():
    summary_text += "requests urgent account verification or immediate action."
elif "win" in text.lower() or "free" in text.lower() or "offer" in text.lower():
    summary_text += "promotes a free offer, prize, or clearance sale."
else:
    summary_text += "seems to be standard communication without urgent requests."

```

```

response["summary"] = summary_text

```

```

return jsonify(response)

```

```

@app.route('/dashboard', methods=['GET'])
def get_dashboard():
    return jsonify({
        "total": total_analyzed,
        "counts": counts
    })

```

"Your Netflix subscription has expired. Please update your payment details at [link] to continue watching your favorite shows.",

"Your Microsoft 365 password will expire soon. Click here to keep your current password.",

"A critical security update is required for your account. Please install it immediately by clicking the link provided.",

"Your Apple ID has been locked for security reasons. To unlock it, please verify your identity here."

]

data = []

for _ in range(50):

 subj = random.choice(safe_subjects)

 body = random.choice(safe_bodies)

 data.append({"text": subj + " - " + body, "label": "Safe"})

for _ in range(25):

 subj = random.choice(spam_subjects)

 body = random.choice(spam_bodies)

 data.append({"text": subj + " - " + body, "label": "Spam"})

for _ in range(25):

 subj = random.choice(phishing_subjects)

 body = random.choice(phishing_bodies)

 data.append({"text": subj + " - " + body, "label": "Phishing"})

random.shuffle(data)

df = pd.DataFrame(data)

df.to_csv('data/emails.csv', index=False)

print('Generated data/emails.csv with 100 rows.')

5. TESTING AND DOCUMENTATION

5.1 Test strategy

- ☐ The system was tested using **Unit Testing, Integration Testing, and System Testing**.
- ☐ Unit testing was performed on individual modules such as preprocessing, classification, and URL analysis.
- ☐ Integration testing was done to ensure all modules work together correctly.
- ☐ System testing was carried out to verify the complete workflow from input to output.
- ☐ The testing was done using different types of emails such as **safe emails, spam emails, and phishing emails**.
- ☐ Testing environment details (Python, Flask, local system, etc.)

5.2 Test Results

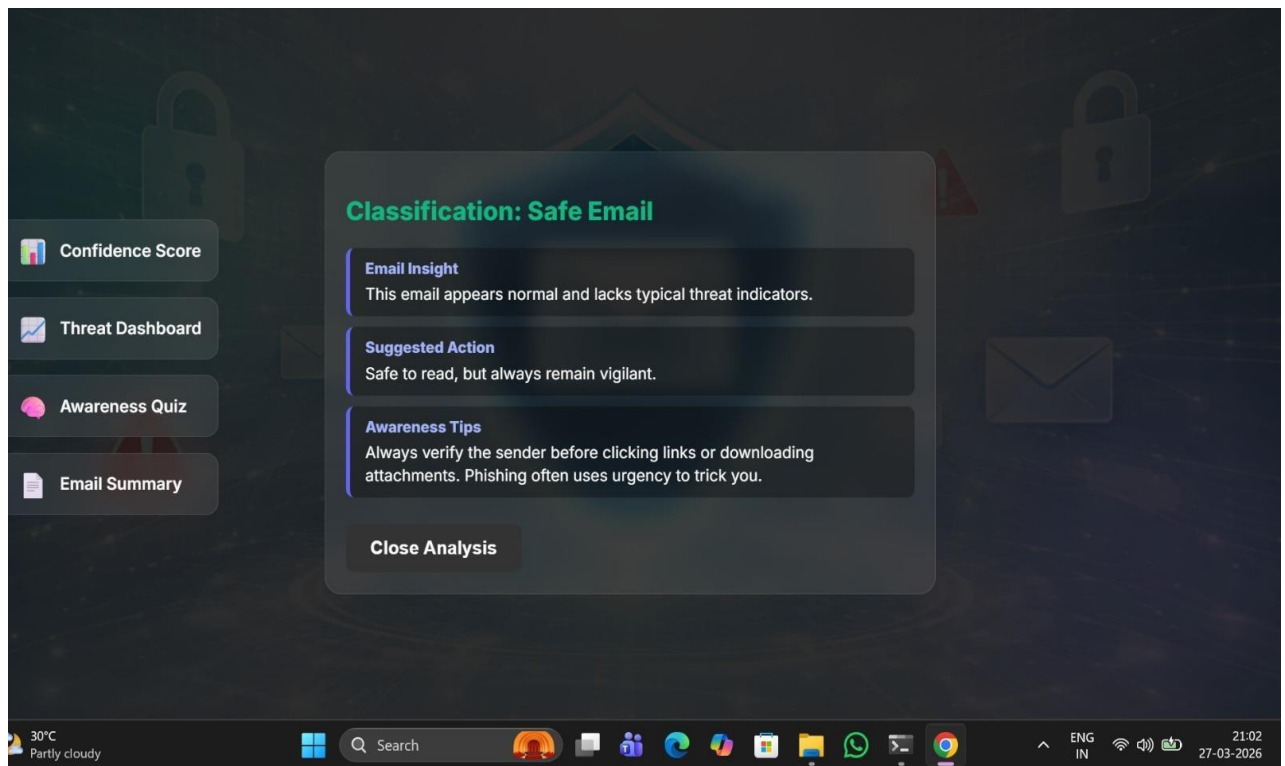


Fig 5.2.1:

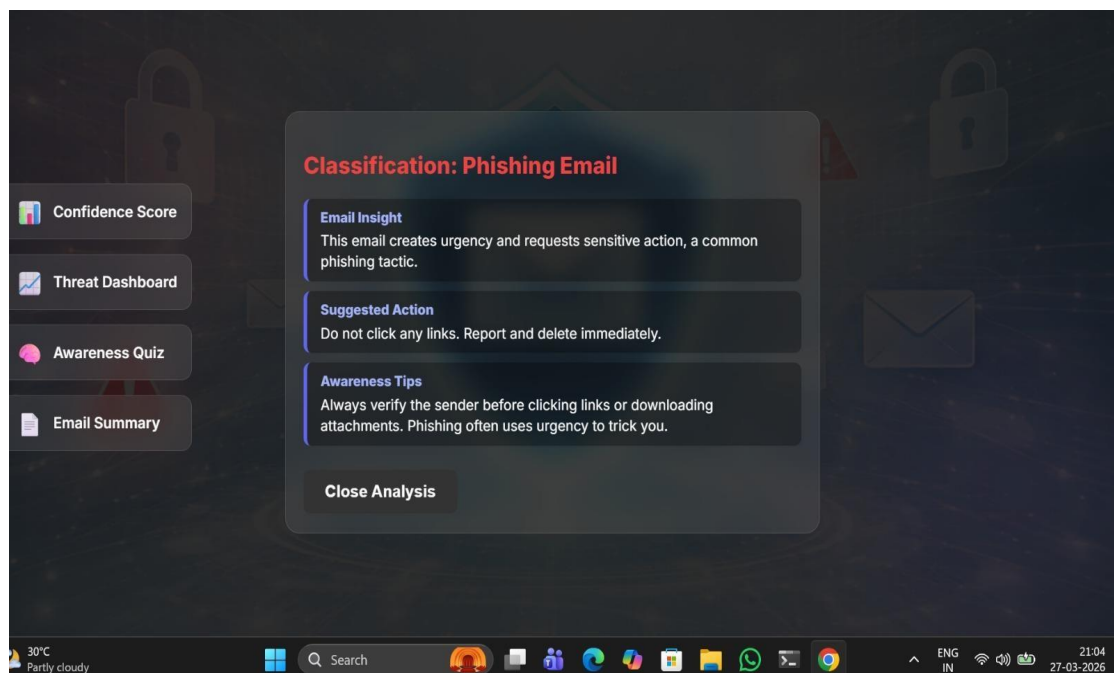
- The system successfully classified emails into Safe/Spam/Phishing.
- The system shows what the Email is about.
- The system gives suggestions and awareness tips.

5.3 Test Documentation

Input Email	Expected Output	Actual Output
Normal message	Safe	Safe
Lottery email	Spam	Spam
Suspicious link	Phishing	Phishing

5.4 User Documenting

- It is a Step-by-step usage:
 1. Open the web application
 2. Enter email subject and body
 3. Click on “Email Analyser”
 4. View classification result
 5. Read suggestions and awareness tips
- Features:
 1. Email classification
 2. Explanation of results
 3. Safety recommendations



6. CONCLUSION

6.1 Evaluation

The development of the email threat detection system represents an important step toward improving cybersecurity in everyday communication. Throughout this project, a structured approach was followed to design, implement, and evaluate a system capable of identifying potentially harmful emails. The system successfully classifies emails into three main categories: Safe, Spam, and Phishing, using machine learning techniques. By analysing key elements such as subject, body content, and embedded URLs, the system demonstrates its ability to detect suspicious patterns and hidden threats with a reasonable level of accuracy.

One of the most significant achievements of this project is the integration of detection with explanation. Unlike traditional systems that simply label emails, this system provides insights into why an email is considered risky. This feature plays a crucial role in improving user understanding and trust. Users are not only informed about the classification result but are also guided with suggestions on how to respond safely. This makes the system more practical and user-centred, especially for individuals who may not have technical knowledge about cybersecurity.

The use of preprocessing techniques and feature extraction methods ensures that the input data is clean and meaningful, which improves the overall performance of the machine learning models. Algorithms such as Logistic Regression and Naïve Bayes have been effectively utilized to achieve reliable classification results. The system also demonstrates efficiency in processing inputs and generating outputs within a short time, making it suitable for real-time usage.

However, like any technological solution, this system also has certain limitations. Its performance largely depends on the quality and diversity of the training dataset. If the system encounters a completely new or highly sophisticated phishing attack that is not represented in the dataset, it may not be able to detect it accurately. Additionally, the system primarily focuses on text-based analysis and may not fully handle threats hidden in images, attachments, or advanced obfuscation techniques. Despite these limitations, the overall performance and functionality of the system indicate that it is a strong foundation for further development.

6.2 Future Enhancement

While the current system provides a reliable solution for email threat detection, there are several opportunities for future improvements that can enhance its performance, scalability, and usability. One of the major areas for enhancement is the adoption of advanced deep learning techniques. Models such as Long Short-Term Memory (LSTM) networks and transformer-based architectures can better understand the context and semantics of email content, leading to improved detection accuracy, especially for complex and evolving phishing attacks.

Another important enhancement is the integration of real-time threat intelligence. By connecting the system to live databases and security feeds, it can continuously update itself with the latest threat patterns, malicious URLs, and phishing strategies. This will enable the system to stay up-to-date and respond effectively to new types of attacks. Additionally, incorporating URL reputation analysis and domain verification techniques can further strengthen the system's ability to detect link-based threats.

The system can also be expanded beyond a standalone web application. It can be developed as a browser extension, email plugin, or mobile application, allowing users to access its features directly within their email platforms. This will improve usability and make the system more accessible in real-world scenarios. Multi-language support can also be added to ensure that users from different regions can benefit from the system.

Another potential improvement is enhancing the user awareness module. The system can include interactive tutorials, training modules, and real-life examples to educate users about common cyber threats. Personalized warnings and adaptive learning features can also be introduced to provide customized guidance based on user behaviour.

Furthermore, the system can be optimized for better performance and scalability by using cloud-based deployment and distributed computing techniques. This will allow it to handle large volumes of data and support multiple users simultaneously. Continuous model training and periodic updates will ensure that the system remains effective over time.

6.3 Technical Skills Learned

The development of this project has provided valuable learning experiences and helped in gaining a wide range of technical skills. One of the key skills acquired is the application of machine learning techniques for real-world problem-solving. Understanding how to preprocess data, extract meaningful features, and train classification models has been an essential part of the project. Practical knowledge of algorithms such as Logistic Regression and Naïve Bayes has also been developed.

The project has also enhanced knowledge in the field of Natural Language Processing (NLP). Techniques such as text cleaning, tokenization, and TF-IDF vectorization were used to process and analyse email content. These skills are highly relevant in handling text-based data and building intelligent systems.

In addition to machine learning, the project has provided experience in software development and system design. Designing a modular system with different components such as data collection, preprocessing, classification, and user interface has improved understanding of system architecture. Working with tools and technologies such as Python, libraries like Scikit-learn, Pandas, and NLTK, and web frameworks like Flask or Django has strengthened programming and development skills.

The project also contributed to improving problem-solving abilities, analytical thinking, and debugging skills. Identifying challenges, testing different approaches, and optimizing the system for better performance were important aspects of the development process. Moreover, working on documentation and presenting the project has improved communication and technical writing skills.

Overall, this project has not only provided a practical solution to a real-world problem but also helped in building a strong foundation in machine learning, cybersecurity, and software development. The knowledge and experience gained through this project will be highly beneficial for future academic and professional endeavours.

7. REFERENCES

1. R. Verma and N. Hossain, “Phishing Email Detection Using Machine Learning,” IEEE, 2019
2. S. Gupta and A. Kumar, “Spam and Malicious Email Detection Using NLP Techniques”2020.
3. J. Ma and L. Huang, “URL-Based Phishing Detection System,” International Journal of Engineering Research & Technology (IJERT), 2019.
4. Alsharnouby and F. O’Kane, “User Awareness Systems for Email Security,” Information Security Journal, 2022.
5. K. Singh and P. Sharma, “Intelligent Email Threat Detection Using Hybrid Models,” 2023.
6. Python Official Documentation - <https://www.python.org>
7. Phishing Email Detection (RCNN model) – THEMIS model (IEEE Access 2019)
8. Detection of URL-based Phishing Attacks Using Machine Learning – IJERT 2019
9. <https://doi.org/10.3390/fi16030086>
10. HELPHED: Hybrid Ensemble Learning Phishing Email Detection (2023)
11. Google Safe Browsing – Phishing & Malware Security: <https://safebrowsing.google.com3>
12. Kaggle Email Spam Dataset (Machine Learning Training Data): <https://www.kaggle.com/datasets>
13. Scikit-learn Machine Learning Documentation: <https://scikit-learn.org>
14. OWASP (Open Web Application Security Project – Phishing Awareness): <https://owasp.org>
15. National Institute of Standards and Technology (NIST Cybersecurity Guidelines): <https://www.nist.gov/cyberframework>

8. BIBLIOGRAPHY

The development of this project was supported by various online resources, research papers, and documentation related to machine learning, natural language processing, and cybersecurity. Standard textbooks and digital learning platforms were also referred to for understanding concepts such as classification algorithms, data preprocessing, and feature extraction techniques.

Online tutorials, developer communities, and open-source documentation played a key role in implementing the system. Platforms like official Python documentation, Scikit-learn guides, and NLP resources helped in gaining practical knowledge for model building and evaluation. In addition, several case studies and research articles on phishing detection and spam filtering were studied to understand real-world challenges and existing solutions.

The knowledge gathered from these sources was carefully analysed and applied to design an effective and user-friendly system. Proper care was taken to ensure originality and avoid direct copying, while still using these references as guidance for understanding concepts and methodologies.

9. APPENDIX

9.1 Screenshots

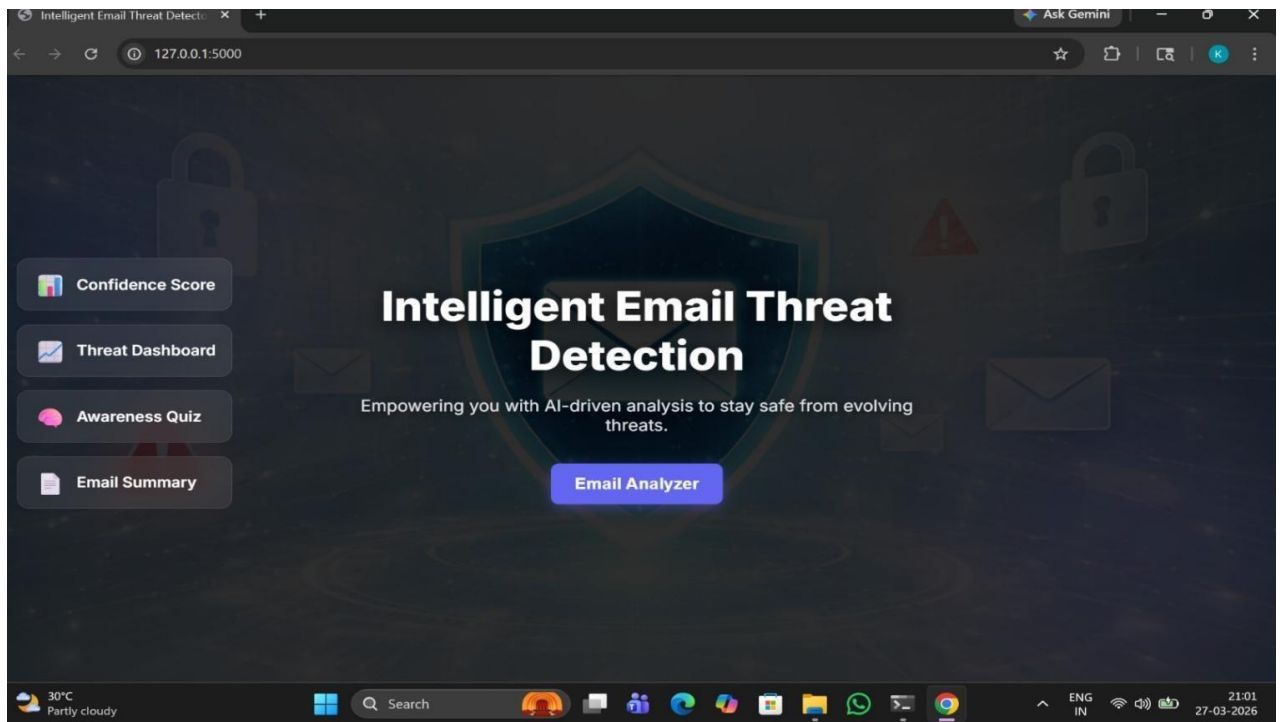


Fig 9.1.1: User Interface

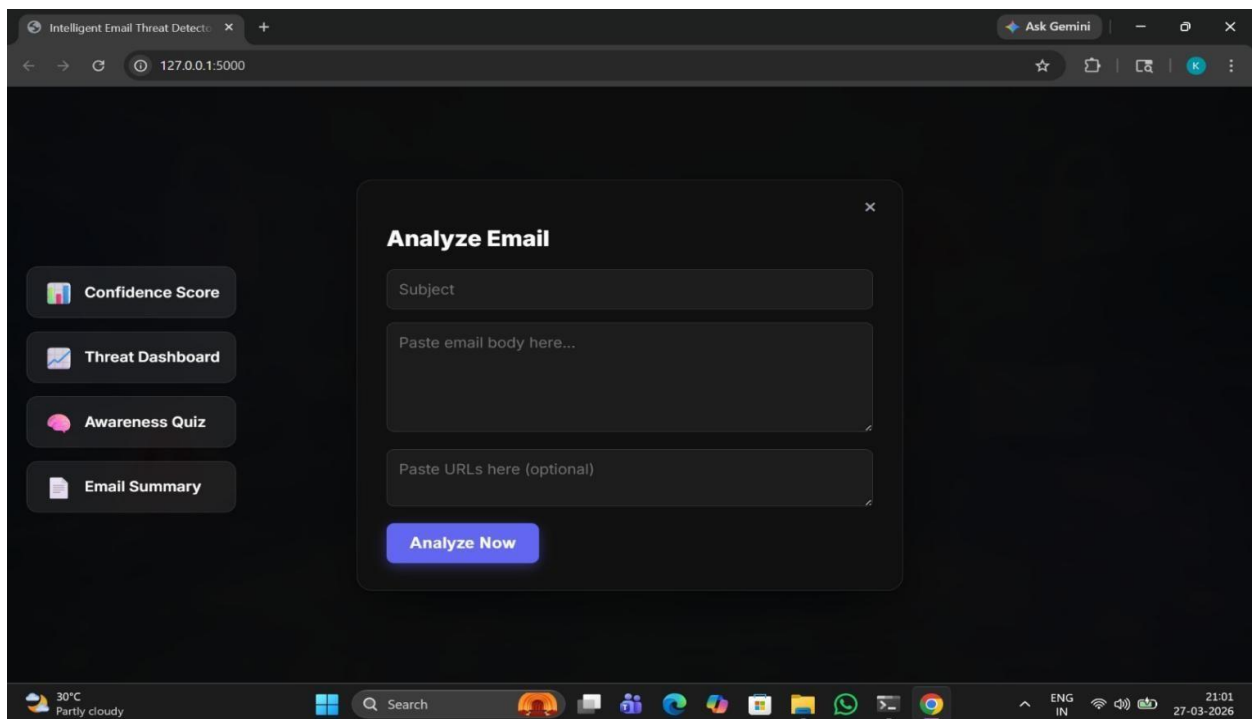


Fig 9.1.2: Analysis page

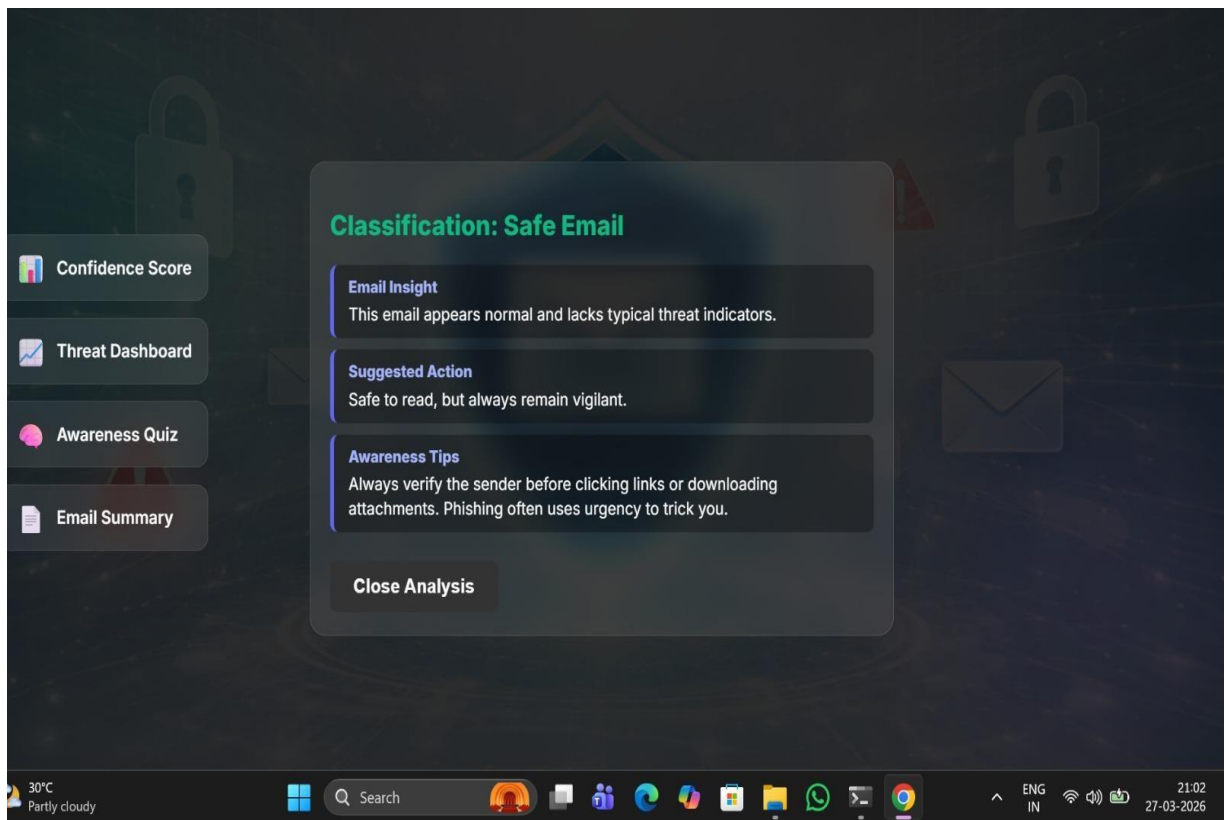


Fig 9.1.3:Classification Results(Safe Email)

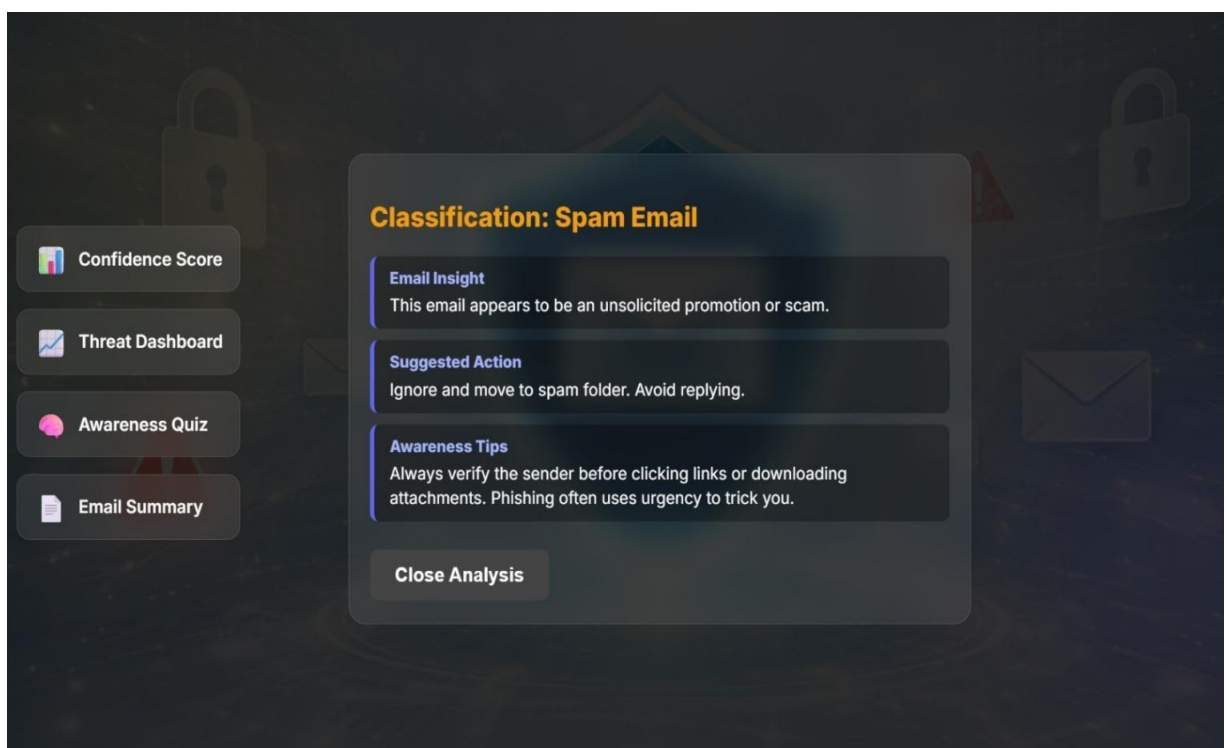


Fig 9.1.4: Classification Results(Spam)

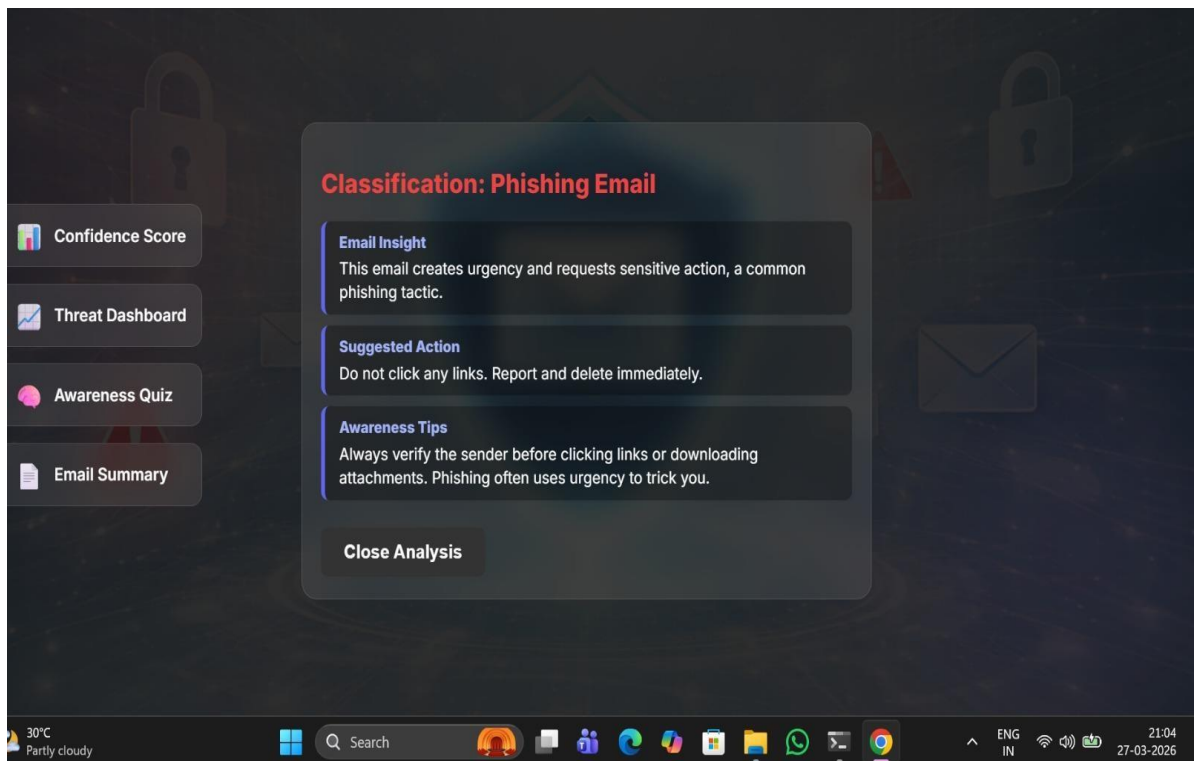


Fig 9.1.5: Classification Results(Phishing Email)

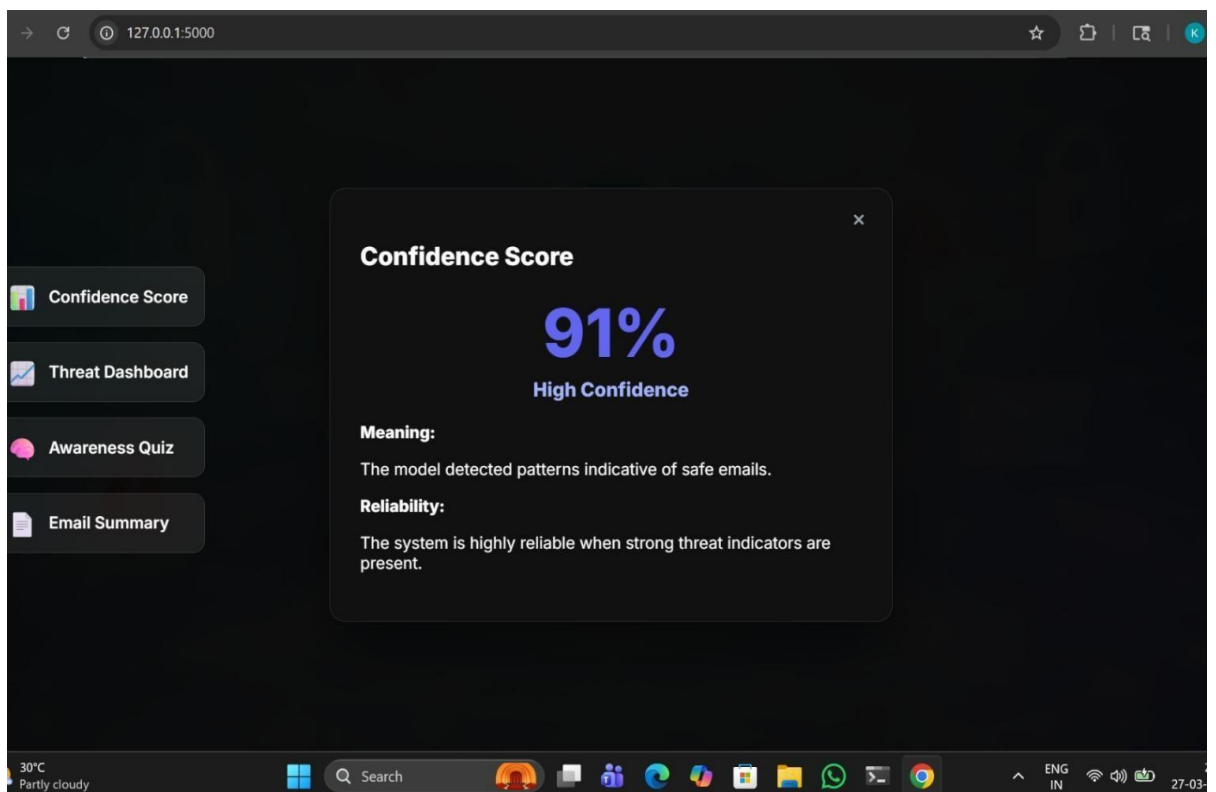


Fig 9.1.6: Confidence Score

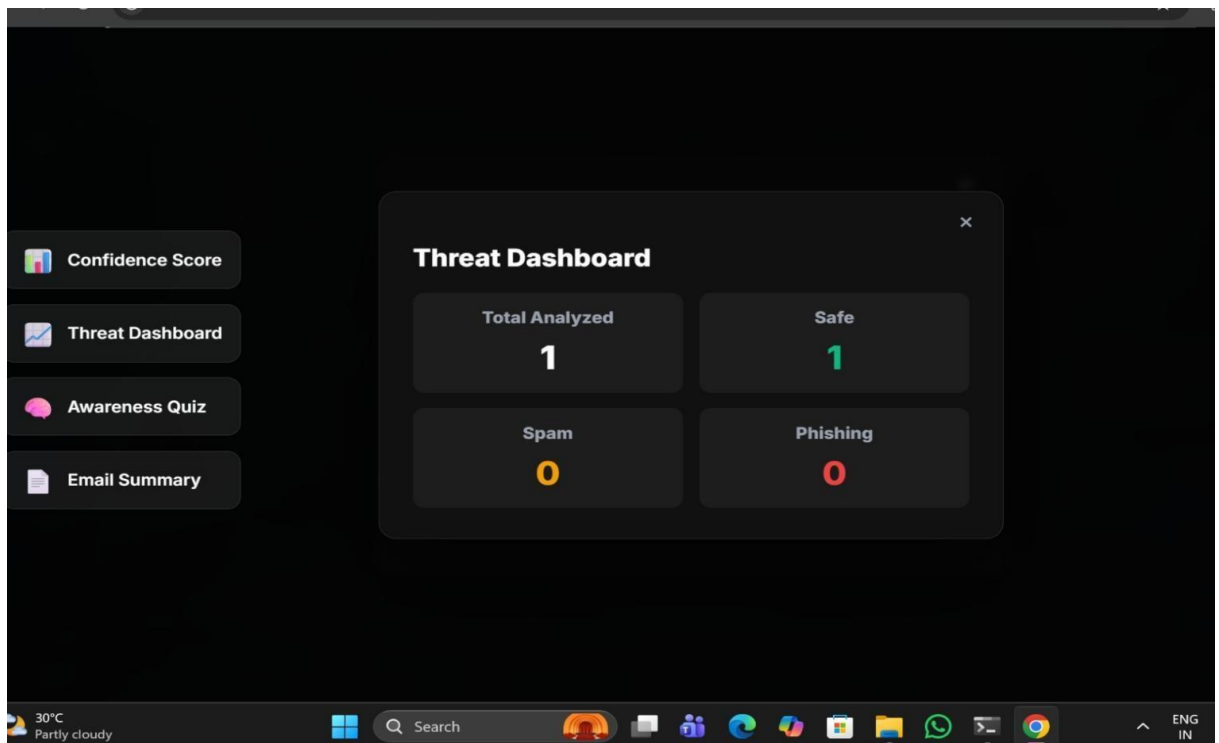


Fig 9.1.7: Threat Dashboard

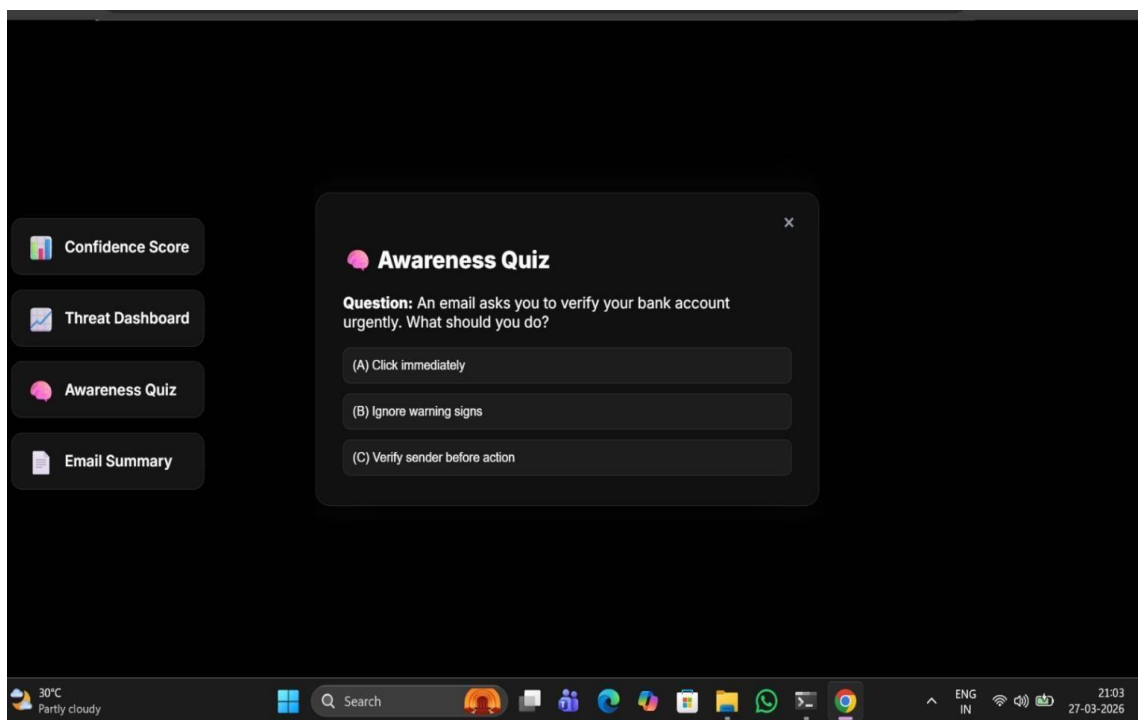


Fig 9.1.8: Awareness Quiz Section

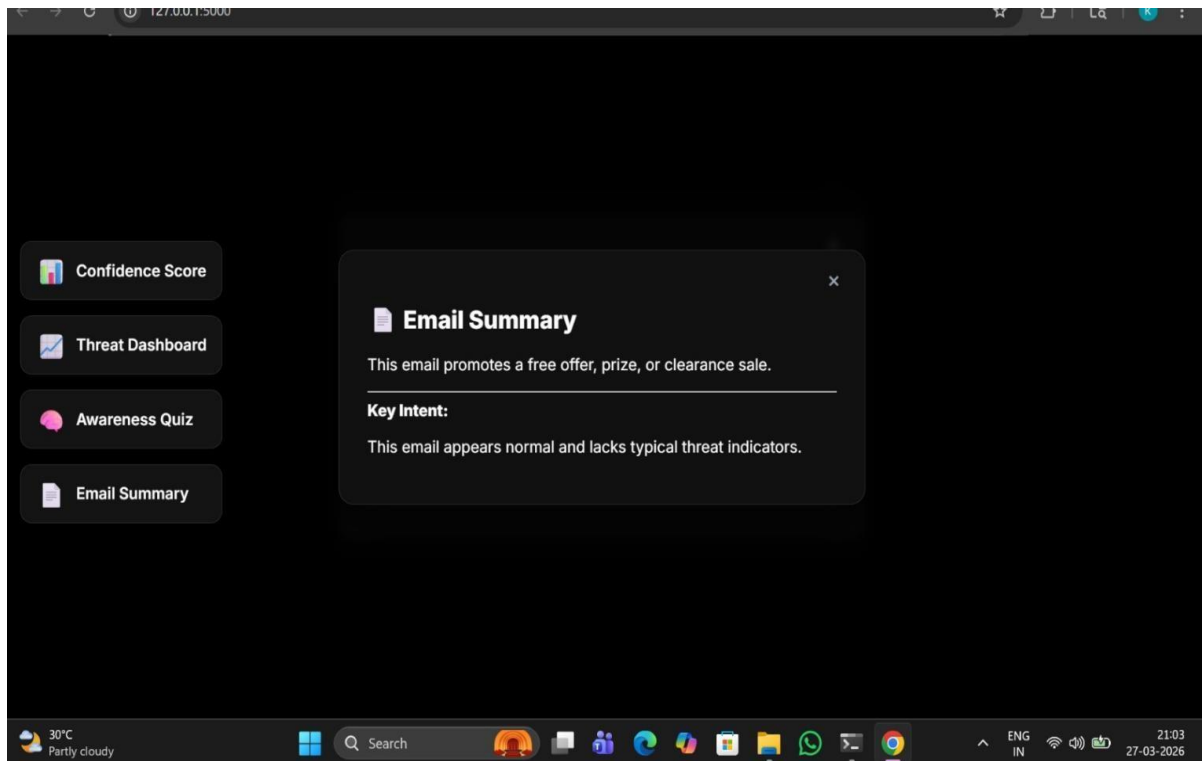


Fig 9.1.9: Email Summary

9.2 Data set

The dataset used in this project consists of a collection of emails categorized into three types: safe, spam, and phishing. The data is obtained from publicly available sources and is used for training and testing the machine learning model.

Each email in the dataset includes key features such as the subject, body content, and URLs, which are analysed to determine the nature of the email. The dataset contains a variety of email types, including normal communication messages, promotional spam emails, and phishing emails designed to mislead users.

This dataset helps the system learn patterns from different types of emails and improves its ability to accurately classify new incoming emails.

9.3 Monthly Reports

The monthly reports section presents the progress of the project during different stages of development. It includes details about the work carried out, such as problem analysis, data collection, system design, implementation, and testing. Each stage explains the tasks completed and the improvements made in the system.

This section helps in tracking the overall development of the project and ensures that all modules are completed in a systematic manner. It also reflects the effort and timeline followed to successfully design and implement the email threat detection system.

The Intelligent Email Threat Detection and User Awareness System project has shown consistent progress over the month, focusing on the development of a robust and user-friendly platform for identifying and mitigating email-based threats. During this period, the system's core functionality—classifying emails into spam, phishing, or safe categories was successfully implemented using machine learning techniques trained on a curated dataset. Enhancements were made to improve detection accuracy by refining preprocessing steps such as tokenization, stop-word removal, and feature extraction.

PA